MANUAL DE REFERÊNCIA TÉCNICA

WALL-8

Arquitetura de hardware, microcódigo e software

Microcomputador discreto de 8 bits

Documento W8-RM-001 | Revisão 1.2

08 de agosto de 2026

Projeto e desenvolvimento: Vinicius Dias de Jesus

Status: baseline de projeto - pré-validação física

Controle do documento

Campo	Valor
Identificação	W8-RM-001
Título	WALL-8 - Manual de Referência Técnica
Revisão	1.1
Data-base	08/08/2026
Autor do projeto	Vinicius Dias de Jesus
Escopo	CPU, memória, I/O, clocks, microcódigo, ISA, interfaces e programação
Estado	Projeto em desenvolvimento; parâmetros elétricos e de timing ainda não certificados
Formato	A4 retrato; a seção do esquema completo foi reservada para anexação posterior

A notação e a nomenclatura deste manual foram normalizadas conforme a arquitetura confirmada pelo autor do projeto. Clock, boot overlay, áudio, display e interface da GPU já aparecem aqui com as definições vigentes, sem manter no corpo do manual as interpretações substituídas.

Como interpretar a revisão

- Comportamento confirmado: aparece diretamente no esquema, no arquivo wall8.cpu ou no gerador de microcódigo.
- Comportamento planejado/reservado: existe endereço, conector ou bit, mas o protocolo final ainda não foi definido.
- Ponto a validar: a lógica está desenhada, porém depende de medição, conteúdo de ROM externo ou confirmação física.

Fontes da baseline

Fonte	Papel normativo nesta revisão
`Hardware_Completo.pdf`	Conexões, componentes, polaridades, conectores e organização física por folhas.
`wall8_apresentacao_arquitetura.md`	Intenção arquitetural, mapa de memória/I/O e descrição dos módulos.
`wall8_instrucoes.md`	Tabela de 76 instruções, bytes, ciclos, flags e exemplos.
`wall8.cpu`	Codificação aceita pelo customasm e ordem dos operandos no binário.
`Gerador_Microcodigo.py`	Fonte definitiva dos 40 sinais de controle e das microsequências.
`7Seg.py`	Fonte das imagens das EPROMs de display e das tabelas HEX/CHAR.
`wall8_placa_video.md`	Contrato CPU-visível da GPU: registradores, comandos, status e handshake.

> **Regra de precedência:** Para opcode e encoding prevalece wall8.cpu; para microsteps prevalece Gerador_Microcodigo; para display prevalece 7Seg.py; para a interface CPU-GPU prevalece o documento da placa de vídeo; para ligação elétrica prevalecem o esquema e as correções confirmadas pelo autor. As definições confirmadas nesta revisão substituem textos antigos incompatíveis.

O Apêndice C registra checksums dos arquivos utilizados. Isso permite saber exatamente a qual baseline este manual se refere mesmo depois que o projeto evoluir.

Sumário

1. Propósito, escopo e status do documento....7
1.1 Objetivo do manual....7
1.2 Baseline e precedência das fontes....7
1.3 Convenções técnicas....7
1.4 Estado de maturidade....8
2. Visão geral da arquitetura....9
2.1 Características principais....9
2.2 Modelo de programação....9
2.3 Organização modular e fluxo de dados....10
3. Alimentação e características elétricas....11
3.1 Entrada de alimentação e proteção....11
3.2 Níveis lógicos e compatibilidade....11
3.3 Desacoplamento, aterramento e carga externa....11
3.4 Parâmetros ainda não caracterizados....12
4. Relógios, temporização, HALT e reset....13
4.1 Clock principal de 16 MHz....13
4.2 Seleção da frequência da CPU....13
4.3 Clock ajustável e clock manual....13
4.4 Distribuição, fanout e HALT....13
4.5 Base de tempo de 1 ms e 500 ms....14
4.6 Reset físico e estados iniciais....14
5. Barramentos e ciclos de máquina....16
5.1 Data Bus de 8 bits....16
5.2 Internal Address Bus de 16 bits....16
5.3 External Address Bus de 16 bits....16
5.4 Ciclo de leitura....16
5.5 Ciclo de escrita....16
5.6 Restrições de temporização....17
6. Registradores e caminho de dados....18
6.1 Registradores visíveis ao programador....18
6.2 Registradores A, B e TMP....18
6.3 PC e IR....18
6.4 MAR e ADDR....18
6.5 PTR e SP....19
6.6 Flags e Step Counter....19
7. Unidade lógica e aritmética....20
7.1 Seleção dos operandos X e Y....20
7.2 Funções da ALU....20

Sumário - continuação

7.3 Semântica das flags....20
7.4 Comparações com e sem sinal....21
8. Sistema de memória e inicialização....22
8.1 Mapa de memória de 64 KiB....22
8.2 RAM principal....22
8.3 BIOS/ROM....22
8.4 Boot overlay e transição para a RAM....23
8.5 Região de cartucho....23
8.6 Aliases, limites e ausência de proteção....23
9. Arquitetura de I/O mapeado em memória....24
9.1 Decode por 74HC138....24
9.2 Espelhos e aliases....24
9.3 Tipos de acesso e efeitos colaterais....24
10. Referência dos registradores de I/O....26
10.1 KEY_DATA - 0xF000....26
10.2 DISPLAY_LOW/HIGH - 0xF100/0xF101....26
10.3 VIDEO_BASE - 0xF200....27
10.4 SYSTEM_STATUS - 0xF300....28
10.5 SOUND_CFG - 0xF400....29
10.6 CPU_CFG - 0xF500....29
10.7 PERIPH_CFG - 0xF501....30
10.8 CART_IO - Y6 e Y7 reservado....30
11. Interfaces externas e pinagem....31
11.1 Teclado - J2....31
11.2 Cartucho - J1....31
11.3 Vídeo/GPU - J4....32
11.4 Alimentação e áudio - J5, J3 e J6....33
11.5 Regras de integração de módulos....33
12. Unidade de controle e microcódigo....34
12.1 Formato do endereço da micro-ROM....34
12.2 Palavra de controle de 40 bits....34
12.3 Valores neutros e polaridade....35
12.4 Codificação das funções da ALU....35
12.5 Fetch comum e encerramento da instrução....35
12.6 Geração e gravação das ROMs....35
12.7 Opcodes não definidos....36
13. Codificação e convenções da ISA....37
13.1 Sintaxe do customasm....37

Sumário - continuação

13.2 Formatos de instrução 37
13.3 Little-endian e cálculo do PC 37
13.4 Ciclos e flags 38
14. Referência completa das 76 instruções 39
14.1 Sistema e controle 39
14.2 Load, store e movimentação 39
14.3 Ponteiro PTR 40
14.4 ALU entre registradores 40
14.5 Comparação, incremento e clear 41
14.6 Jumps, pilha e subrotinas 41
14.7 ALU com valor imediato 42
15. Pilha, subrotinas e práticas de programação 43
15.1 Organização da pilha 43
15.2 CALL, RET, PUSH e POP 43
15.3 Convenção de chamada recomendada 43
15.4 Shadow de registradores write-only 44
15.5 Polling de timers e periféricos 44
16. Exemplos de programação 45
16.1 Sequência segura de boot 45
16.2 Leitura do teclado 45
16.3 Polling do timer 45
16.4 Display de quatro dígitos 45
16.5 Geração de som 46
16.6 Percurso sequencial com PTR+ 46
17. BIOS, diagnóstico e fluxo de validação 47
17.1 Responsabilidades mínimas da BIOS 47
17.2 Ordem de diagnóstico sugerida 47
17.3 Verificação do microcódigo 47
18. Estado do projeto, limitações e notas de implementação 49
18.1 Recursos confirmados, reservados e futuros 49
18.2 Checklist antes da fabricação 49
19. Apêndices - Anexos 50
Apêndice A - Microsequências por opcode 51
Apêndice B - Glossário dos sinais de controle 56
Apêndice C - Baseline e checksums 58
Apêndice D - Histórico de revisões 59
Anexo A — Esquema elétrico completo 60

1. Propósito, escopo e status do documento

1.1 Objetivo do manual

Este manual apresenta o WALL-8 como uma arquitetura completa e programável, e não apenas como um conjunto de folhas de esquema. O objetivo é permitir que uma pessoa compreenda o comportamento elétrico e lógico da CPU, programe a ISA, gere as ROMs de microcódigo, integre módulos externos e execute uma validação ordenada da placa.

O texto cobre o núcleo discreto de 8 bits, a hierarquia de barramentos, os registradores, a ALU, o mapa de memória, o boot overlay, o I/O mapeado, os conectores, as 76 instruções e as microsequências. O esquema elétrico integral não é reproduzido: a Seção 19 reserva o ponto exato em que as 17 folhas deverão ser anexadas.

1.2 Baseline e precedência das fontes

A baseline usa o esquema Revisão 1.0 datado de 07/08/2026 e os arquivos de arquitetura, ISA e microcódigo fornecidos em 08/08/2026. Como o projeto está em evolução, um nome de sinal impresso pode refletir uma versão anterior, enquanto a conexão efetiva e o software já representam a revisão atual.

Assunto	Fonte de autoridade	Critério
Opcode e bytes	wall8.cpu	A regra aceita pelo montador define o binário.
Ciclos e flags	Gerador + referência de instruções	A sequência de T0 até STEP_RESET define os ciclos.
Polaridade elétrica	Esquema + tabela de controle	A bolha, o prefixo '/' e o nível ativo são avaliados em conjunto.
Mapa de I/O	Decode físico	A11 ignorado cria espelhos; A0 cria aliases internos.
Recursos futuros	Arquitetura	São descritos como reservados, sem inventar protocolo.

1.3 Convenções técnicas

Convenção	Significado
`0x1234 ou $1234`	Número hexadecimal; a sintaxe de assembly usa '$'.
`#$55`	Operando imediato de 8 bits.
`bit 0`	Bit menos significativo (LSB).
`/SINAL`	Sinal ativo em nível baixo. No gerador, o nome pode aparecer sem a barra, mas a polaridade está na tupla de controle.
`LL HH`	Endereço de 16 bits codificado em little-endian: byte baixo antes do byte alto.
`T0...T15`	Microsteps do contador de 4 bits.
`assertar`	Colocar um sinal em seu nível ativo, independentemente de ele ser alto ou baixo.
`NC / reservado`	Não conectado ou sem função definida; escrever zero é a prática recomendada.

1.4 Estado de maturidade

> **Status:** A arquitetura lógica, a ISA, o microcódigo, os conteúdos lógicos das ROMs de display e o contrato CPU-visível da GPU estão definidos. A placa física, especialmente a operação a 16 MHz, consumo total, margens elétricas e tempos de módulos externos, ainda precisa de caracterização em bancada.

Não existem, nesta baseline, interrupções, DMA, barramento serial, mecanismo de WAIT/READY, proteção de memória ou detecção de stack overflow. Esses itens devem ser tratados como fora de escopo, e não como comportamento implícito.

2. Visão geral da arquitetura

2.1 Características principais

Característica	Valor / implementação
Processador	CPU discreta de 8 bits com lógica 74HC/74HCT; U117 deve ser considerado 74HC541.
Espaço de endereço	64 KiB, endereços de 16 bits.
Data Bus	8 bits, tri-state, compartilhado por registradores, memória e I/O.
Barramentos de endereço	16 bits interno e 16 bits externo, separados pelo MAR.
Controle	5 EPROMs 27C512, 8 bits cada: palavra de controle de 40 bits.
Microstep	Contador de 4 bits, T0 a T15.
Flags	Z, C, N e V.
Memória base	32 KiB RAM, BIOS de 8 KiB em operação normal ou ROM contínua de 40 KiB no overlay, 20 KiB de cartucho e 4 KiB de I/O.
Clock da CPU	16 MHz a 125 kHz selecionável; código 000 faz bypass direto do cristal, além de debug ajustável e manual.
ISA	76 opcodes definidos, instruções de 1 a 4 bytes.
Pilha	16 bits, crescimento descendente, CALL/RET e PUSH/POP.
I/O	Mapeado em memória; teclado, display, timers/status, som, vídeo e configuração.

2.2 Modelo de programação

O programador opera principalmente com A e B, ambos de 8 bits. PTR fornece endereçamento indireto e pós-incremento; SP dá suporte à pilha; PC e flags controlam o fluxo. IR, MAR, ADDR, TMP e o Step Counter pertencem à microarquitetura e não possuem instrução geral de leitura.

Recurso	Largura	Visibilidade	Uso principal
`A / ACC`	8	Direta	Acumulador e operando X da ALU.
`B`	8	Direta	Segundo operando, contador e auxiliar.
`PTR`	16	Indireta	Acesso sequencial à memória.
`SP`	16	Indireta	Pilha e endereços de retorno.
`PC`	16	Implícita	Próxima instrução e saltos.
`Z/C/N/V`	4	Via jumps	Resultado zero, carry, negativo e overflow.
`TMP/ADDR/MAR/IR`	8/16	Microcódigo	Temporários e interface com memória.

2.3 Organização modular e fluxo de dados

O projeto separa o endereço em dois domínios. PC, ADDR, PTR e SP dirigem o Internal Address Bus; o MAR captura esse valor e dirige o External Address Bus. Em paralelo, o Data Bus move opcodes, operandos, resultados e bytes de endereço.

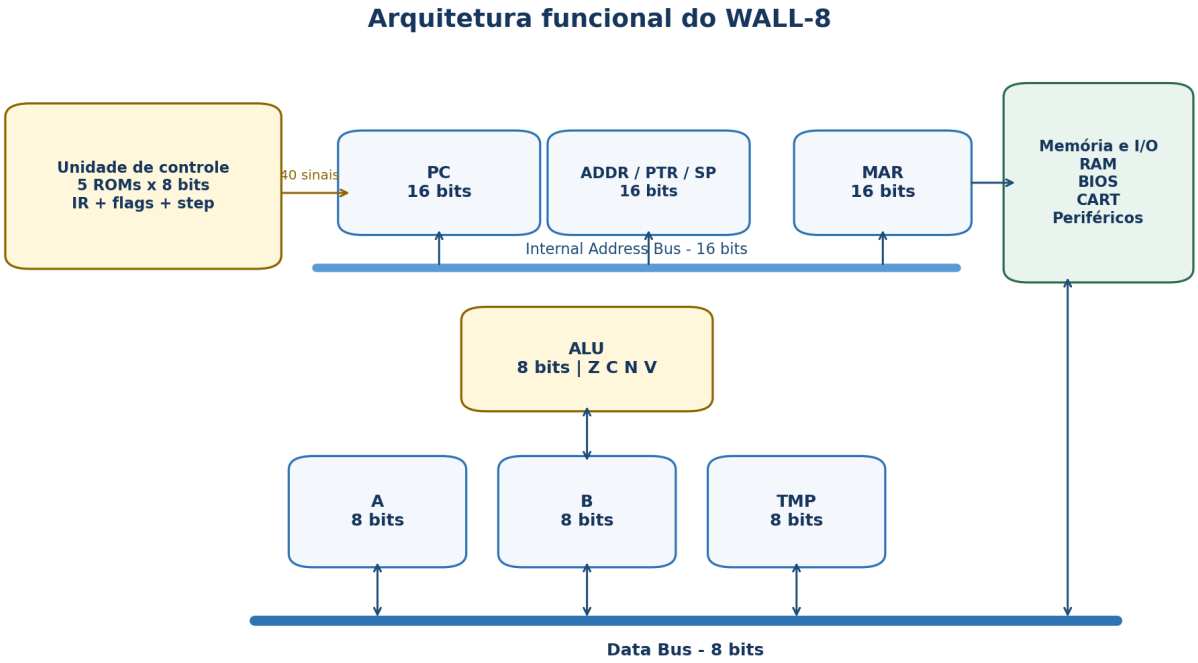


Figura 1 - Blocos funcionais e os três barramentos do WALL-8.

> **Regra de contenção:** Em cada barramento tri-state apenas uma fonte deve ser habilitada por microstep. A palavra de controle foi construída para impedir que dois registradores ou a memória dirijam simultaneamente o mesmo barramento.

3. Alimentação e características elétricas

3.1 Entrada de alimentação e proteção

A placa utiliza uma única alimentação nominal de +5 V referenciada a GND. Não há regulador principal desenhado: a fonte externa deve entregar 5 V regulados. A entrada J5 passa por um fusível de 3 A e possui um diodo 1N5408 em proteção de polaridade/crowbar conforme a folha Unit Control.

Parâmetro	Símbolo	Valor	Observação
Tensão nominal	VCC	5,0 V	Comum à lógica, EPROM, RAM, timers e áudio.
Faixa de uso recomendada	VCC	4,75 a 5,25 V	Faixa conservadora até a caracterização completa; não é limite absoluto certificado.
Referência	GND	0 V	Todos os módulos externos devem compartilhar o mesmo GND.
Fusível de entrada	FU1	3 A	Proteção; não representa o consumo nominal da placa.
Conector	J5	2 vias	Pino 1 GND; pino 2 +5 V através de FU1.

> **Não exceder:** Não aplique 9 V, 12 V ou tensão de bateria diretamente em J5. A placa espera 5 V já regulados.

3.2 Níveis lógicos e compatibilidade

Os barramentos externos são sinais digitais de 5 V. Módulos de cartucho, vídeo e teclado devem usar lógica compatível com 5 V ou tradução de nível. Uma saída de 3,3 V não deve ser considerada automaticamente compatível com uma entrada 74HC alimentada em 5 V, e as entradas dos módulos de 3,3 V não devem receber 5 V sem proteção.

Interface	Direção típica	Requisito
EXBUS[15:0]	CPU -> módulo	Entrada de 5 V no módulo; endereço válido durante /SEL e OE/WE.
DBUS[7:0]	Bidirecional	Tri-state obrigatório; nunca dirigir durante leitura de outra fonte.
/MEMORY_OE, /MEMORY_WE, /SEL	CPU -> módulo	Ativos em nível baixo, compatíveis com 5 V.
/DETECT	Módulo -> CPU	Saída open-collector/open-drain ou contato para GND é o método mais seguro; a CPU possui pull-up.
KEY_READY	Teclado -> CPU	Alto indica byte pronto; há pull-down na placa.

U117, que coloca o teclado no Data Bus, é um 74HC541 alimentado em 5 V. Suas saídas permanecem em alta impedância fora dos ciclos de leitura de KEY_DATA.

3.3 Desacoplamento, aterramento e carga externa

O esquema distribui capacitores de 100 nF próximos aos circuitos integrados e inclui bancos de 470 uF/16 V na alimentação. Os clocks usam resistores série de 33 ohms em cada ramo de distribuição. Essas medidas reduzem ruído local, mas não dispensam planos de GND contínuos, caminhos de retorno curtos e medição de ringing na placa física.

- Use GND e +5 V em mais de um pino nos conectores de 40 vias, conforme a pinagem.
- Não alimente periféricos de corrente elevada pelo barramento sem incluir sua carga no orçamento da fonte e do fusível.
- Mantenha os capacitores de 100 nF fisicamente próximos aos pinos VCC/GND de cada CI.
- Os resistores de 1 k dos LEDs e os buffers dedicados evitam que os indicadores carreguem diretamente os registradores.

3.4 Parâmetros ainda não caracterizados

Item	Situação atual	Medição necessária
Consumo total	Não informado	Corrente em idle, execução, LEDs ligados, áudio e módulos externos.
Frequência máxima	16 MHz selecionável	Margem em temperatura/tensão e pior caminho de propagação.
Níveis VIH/VIL do sistema	Família HC/HCT	Compatibilidade de módulos externos e ruído no barramento.
Tempos de memória/I/O	Sem WAIT	Setup/hold de RAM, EPROM, cartucho e GPU.
Áudio	Funcional no esquema	Nível no P2, potência, THD, estabilidade e temperatura do TDA2822M.
Hot-plug/ESD	Não implementado	Não conectar ou remover módulos energizados.

4. Relógios, temporização, HALT e reset

4.1 Clock principal de 16 MHz

O oscilador principal usa um cristal de 16 MHz, portas 74HCU04, condicionamento por 74HC14, resistor de realimentação de 2,2 Mohms, resistor série de 680 ohms e capacitores de 22 pF. O 74HC151 recebe o clock direto e as sete divisões sucessivas produzidas pelo 74HC4040.

> **Código 000:** CLK_B[2:0] = 000 seleciona o bypass do divisor: o clock principal da CPU é o próprio sinal de 16 MHz do cristal.

4.2 Seleção da frequência da CPU

CLK_B2..0	Clock da CPU	Período	CPU_CFG com RAM ativa
000	16 MHz	62.5 ns	0x01
001	8 MHz	125 ns	0x11
010	4 MHz	250 ns	0x21
011	2 MHz	500 ns	0x31
100	1 MHz	1000 ns	0x41
101	500 kHz	2000 ns	0x51
110	250 kHz	4000 ns	0x61
111	125 kHz	8000 ns	0x71

Os bits de seleção ocupam CPU_CFG[6:4], com CLK_B0 no bit 4. O bit 0 deve ser 1 para deixar a RAM visível em 0x0000-0x7FFF. Assim, 0x01 significa RAM ativa e clock de 16 MHz; 0x11 seleciona 8 MHz; 0x41 seleciona 1 MHz.

4.3 Clock ajustável e clock manual

Uma metade do NE556 forma o clock ajustável de debug, controlado por RV1. A outra metade gera um pulso limpo a partir do botão BUT1. O seletor rotativo SW1 escolhe CLOCK_MAIN, CLOCK_DEBUG_AJUST ou CLOCK_MANUAL antes da lógica de HALT.

> **Frequência de debug:** A faixa mínima/máxima do modo ajustável não está formalmente caracterizada na documentação. Ela deve ser medida nos extremos de RV1 antes de ser publicada como especificação.

4.4 Distribuição, fanout e HALT

O clock selecionado passa por uma porta controlada por /HALT. Depois, seis portas 74HC04 fornecem ramos independentes com resistores série de 33 ohms: CLOCK_PC, CLOCK_SP, CLOCK_PTR, CLOCK_ADDMAR, CLOCK_PeRA e CLOCK_RB-IR-TM-FL-ST. Os ramos têm a mesma frequência e existem para fanout e integridade de sinal, não como domínios assíncronos.

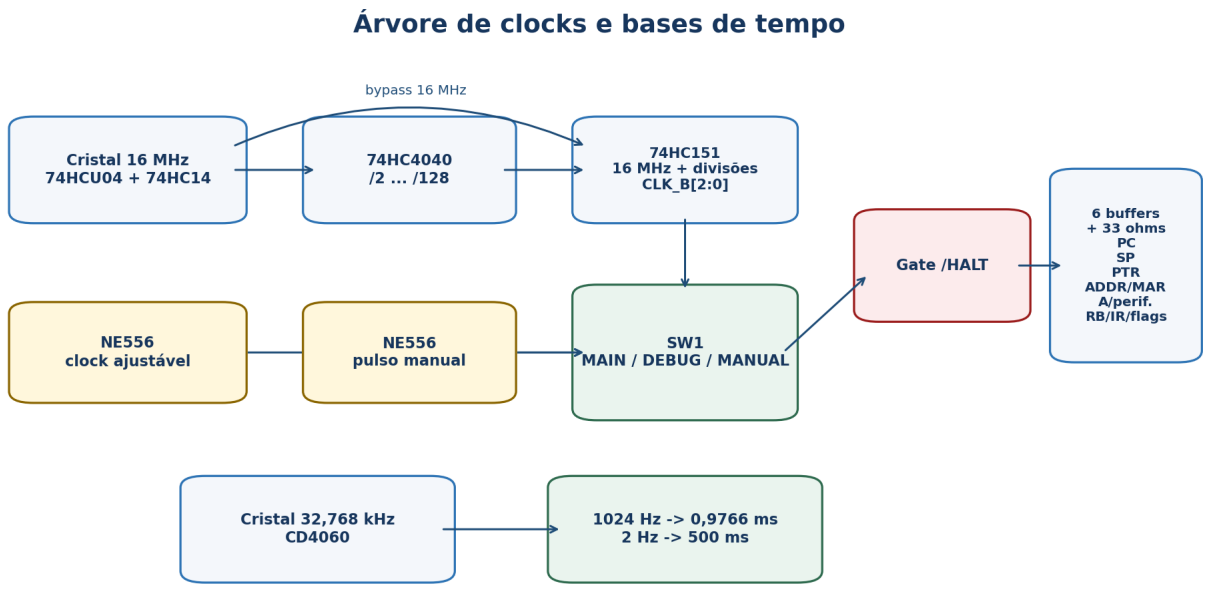


Figura 2 - Fontes, seleção e distribuição do clock.

HLT assertará /HALT no microstep T2 e interromperá a passagem de clock. Não existe instrução de resume nem interrupção: a saída prevista é o reset físico.

4.5 Base de tempo de 1 ms e 500 ms

Um cristal de 32,768 kHz e um 74HC4060 produzem duas bases de tempo independentes do clock da CPU. A saída nomeada TIMER_1MS é nominalmente 1024 Hz, com período de 0,9765625 ms. TIMER_500MS opera a 2 Hz, com período de 500 ms.

Cada sinal aciona um flip-flop 74HC74 cujo D está em nível alto. O primeiro pulso após a limpeza seta o respectivo bit e ele permanece alto até qualquer escrita na página TIMER_SEL. Portanto os bits são flags latched, não ondas quadradas lidas diretamente e não contadores de eventos.

4.6 Reset físico e estados iniciais

O hard reset usa R154 de 100 kohms, C92 de 10 uF, BUT2 e portas Schmitt. A constante de tempo nominal é da ordem de 1 s. O circuito gera /STEP_MR, /PC_MR, Reset_Config e controla /ROM_MICRO_OE para manter as saídas das ROMs de controle desabilitadas durante a inicialização.

Elemento	Após hard reset	Observação
PC	0x0000	Reset assíncrono dos 74HC161.
Step Counter	T0	Master reset do 74HC161.
CPU_CFG	0x00	Reset_Config habilita o latch com Data Bus puxado para baixo.
PERIPH_CFG	0x00	HEX e LEDs ativos pelos sinais de nome ativo-baixo.
Boot overlay	Ativo	/BOOT_ROM_EN = 0.
Clock selecionado	16 MHz	CLK_B[2:0] = 000, bypass direto do cristal.
A, B, TMP, ADDR, MAR, PTR, SP, flags	Indefinido	Devem ser inicializados quando relevantes.
SOUND_CFG e display	Indefinido	A BIOS deve escrever valores seguros imediatamente.

Elemento	Após hard reset	Observação
`Flags de timer`	`Não garantido`	A BIOS deve escrever em 0xF300 para limpar.

> **Sequência mínima:** O primeiro código executado não deve depender de A, B, SP, PTR ou flags. Inicialize o áudio e o display, limpe os timers, carregue SP e apenas depois execute rotinas que dependam desses estados.

5. Barramentos e ciclos de máquina

5.1 Data Bus de 8 bits

DBUS[7:0] transporta opcodes, imediatos, dados de RAM/ROM/cartucho/I/O, resultados da ALU, valores de A/B/TMP e bytes do PC usados por CALL. A faixa natural é 0x00-0xFF e as operações aritméticas são módulo 256.

Possíveis fontes	Possíveis destinos
Memória/I/O via /MEMORY_OE	IR, A, B, ADDR_L/H, PTR_L/H, SP_L/H, TMP
A, B, TMP	Memória/I/O via /MEMORY_WE e outros registradores
ALU	A, B e Data Bus
PC low / PC high	Memória da pilha durante CALL

5.2 Internal Address Bus de 16 bits

INBUS[15:0] move endereços dentro da CPU. PC, ADDR, PTR e SP possuem buffers 74HC245 para esse barramento. O MAR é o destino normal; PC também pode carregar o endereço fornecido por ADDR durante jumps e RET.

```
PC / ADDR / PTR / SP  ->  Internal Address Bus  ->  MAR
ADDR                  ->  Internal Address Bus  ->  PC
```

5.3 External Address Bus de 16 bits

EXBUS[15:0] é dirigido pelo MAR quando /MAR_OUT está ativo. Ele alimenta RAM, BIOS, decode de memória, conectores de cartucho/vídeo e o sub-decode de I/O. Pull-downs de 68 kohms definem nível baixo quando os drivers estão em alta impedância.

5.4 Ciclo de leitura

Uma leitura absoluta separa a busca do endereço em dois bytes e a leitura final do dado. LDA $1234, por exemplo, executa:

```
T2: PC_OUT + MAR_LOAD
T3: MAR_OUT + MEMORY_OE + ADDR_L_LOAD + PC_INC
T4: PC_OUT + MAR_LOAD
T5: MAR_OUT + MEMORY_OE + ADDR_H_LOAD + PC_INC
T6: ADDR_OUT + MAR_LOAD
T7: MAR_OUT + MEMORY_OE + RA_LOAD
T8: STEP_RESET
```

A leitura via PTR é menor porque o endereço de 16 bits já está pronto: PTR_OUT carrega MAR em T2 e o dado é capturado em T3.

5.5 Ciclo de escrita

As escritas usam um microstep de setup antes de assertar /MEMORY_WE. Em STA $addr, T7 coloca MAR no barramento externo e A no Data Bus; T8 mantém os dois estáveis e baixa /MEMORY_WE. O mesmo padrão é usado por STB, STI, PUSH e CALL.

```
T7: MAR_OUT + RA_OUT                ; setup de endereço e dado
T8: MAR_OUT + RA_OUT + MEMORY_WE    ; pulso de escrita
```

> **Dados de STI:** STI busca um quarto byte, guarda o imediato em TMP e somente então dirige TMP no Data Bus. A e B não são alterados.

5.6 Restrições de temporização

Não existe entrada WAIT, READY de memória ou extensão automática de ciclo. Toda RAM, EPROM ou lógica externa deve responder dentro do microstep definido pelo clock selecionado. Em 16 MHz cada microstep dura 62,5 ns; desse orçamento ainda devem ser descontados atrasos de contador, ROM de controle, buffers, decode e setup do registrador de destino.

Clock	Microstep	Instrução de 4 ciclos	CALL de 16 ciclos
16 MHz	62,5 ns	0,25 us	1,00 us
8 MHz	125 ns	0,50 us	2,00 us
4 MHz	250 ns	1,00 us	4,00 us
2 MHz	500 ns	2,00 us	8,00 us
1 MHz	1 us	4,00 us	16,00 us
125 kHz	8 us	32 us	128 us

6. Registradores e caminho de dados

6.1 Registradores visíveis ao programador

Registrador	Bits	Carga / alteração	Leitura / uso
A	8	LDA, MOV, ALU, POP	ALU, STA, PUSH
B	8	LDB, MOV, ALU, POP	ALU, STB, PUSH
PTR	16	PTR, INC PTR, DEC PTR, PTR+	Endereçamento indireto
SP	16	SP, PUSH/POP, CALL/RET	Pilha
PC	16	Fetch, JMP/Jcc, CALL/RET	Busca de instrução
Flags	4	Operações da ALU	JZ/JNZ/JC/JNC/JN/JP/JV/JNV

Não há instrução para copiar diretamente PTR, SP, PC ou o registrador de flags para A/B. Quando um software precisa manter esses valores, deve conservar uma cópia em RAM ou construir uma convenção própria.

6.2 Registradores A, B e TMP

A e B usam latches 74HC377 e buffers 74HC245. Entram diretamente na ALU. TMP também é um 74HC377 com entrada/saída no Data Bus, mas não é operando físico da ALU.

Nas formas imediatas sobre A, B é salvo em TMP, recebe o imediato, participa da ALU e depois é restaurado. Nas formas sobre B, A é preservado da mesma maneira. CMP e TEST atualizam flags sem gravar o resultado.

```
ADD A, #$05:
  T2  TMP <- B
  T3/T4  B <- byte imediato; PC++
  T5  A <- A + B; flags <- resultado
  T6  B <- TMP
  T7  fim
```

6.3 PC e IR

PC é um contador síncrono de 16 bits construído com quatro 74HC161. Pode incrementar, carregar do Internal Address Bus, ser resetado a zero e colocar seus bytes low/high no Data Bus durante CALL. IR é um latch de 8 bits que captura o opcode no fetch e alimenta A0-A7 das cinco ROMs de controle.

```
T0: PC -> MAR
T1: MEM[MAR] -> IR; PC <- PC + 1
```

6.4 MAR e ADDR

MAR é o registrador de endereço de memória: recebe 16 bits do barramento interno e, quando habilitado, dirige o barramento externo. ADDR é formado por dois bytes lidos do fluxo de instrução e fornece o endereço absoluto usado por load/store, jumps, CALL e RET.

> **Little-endian:** ADDR_L é carregado primeiro e ADDR_H depois. A instrução LDA $1234 é codificada como 12 34 12.

6.5 PTR e SP

PTR e SP usam contadores 74HC191 em cascata, permitindo carga paralela de 16 bits e contagem ascendente/descendente. PTR participa do endereçamento indireto. SP usa pre-decremento no push e pós-incremento no pop.

Operação	Sequência lógica
`INC PTR`	`PTR <- PTR + 1`
`DEC PTR`	`PTR <- PTR - 1`
`[PTR+]`	`Acessa MEM[PTR] e somente depois incrementa PTR.`
`PUSH`	`SP <- SP - 1; MEM[SP] <- dado.`
`POP`	`dado <- MEM[SP]; SP <- SP + 1.`

6.6 Flags e Step Counter

O registrador de flags armazena Z, C, N e V em um 74HC377. Os quatro bits entram em A8-A11 das micro-ROMs. O Step Counter de 4 bits fornece A12-A15 e avança a cada clock até que /STEP_RESET carregue zero.

Como as flags fazem parte do endereço da micro-ROM, um mesmo opcode pode selecionar microinstruções diferentes sem uma unidade condicional separada. Os jumps leem seus operandos em todas as situações, mas só assertam PC_LOAD no T6 quando a condição é verdadeira.

7. Unidade lógica e aritmética

7.1 Seleção dos operandos X e Y

A ALU recebe A e B através de multiplexadores. ALU_AB_SWAP = 0 seleciona X=A e Y=B; ALU_AB_SWAP = 1 seleciona X=B e Y=A. O swap é indispensável para SUB B,A, CMP B,A e operações unárias sobre B. Para AND/OR/XOR a ordem não altera o resultado.

7.2 Funções da ALU

MUX2..0	ALU_SUB	ALU_NOT_X	Função
000	0	X	X + Y
000	1	X	X - Y
001	X	X	X AND Y
010	X	X	X OR Y
011	X	0	X XOR Y
011	X	1	NOT X
100	X	X	SHR X; C <- X0
101	X	X	SHL X; C <- X7
110	0	X	X + 1
110	1	X	X - 1
111	X	X	0x00

SHR é lógico e insere zero no bit 7. SHL insere zero no bit 0. Não existe shift aritmético nem rotação por carry nesta ISA.

7.3 Semântica das flags

Flag	Definição	Detalhes
Z	Resultado = 0x00	Em TEST, considera o resultado mascarado; em CLR fica 1.
C	Carry / not borrow / bit deslocado	ADD: carry-out. SUB/CMP: 1 significa sem borrow. SHL/SHR: bit removido.
N	Resultado[7]	Indica o bit de sinal em complemento de dois, mas sozinho não resolve toda comparação signed.
V	Overflow signed	Válido em ADD/SUB/INC/DEC; a ALU força 0 nas funções lógicas, shifts e ZERO.

Para soma de 8 bits, V = NOT(X7 XOR Y7) AND (R7 XOR X7). Para subtração, V = (X7 XOR Y7) AND (R7 XOR X7). INC gera overflow em 0x7F -> 0x80; DEC em 0x80 -> 0x7F.

7.4 Comparações com e sem sinal

CMP executa uma subtração apenas nas flags. Em comparação unsigned, Z=1 significa igualdade e C=1 após A-B significa A >= B. Em signed, menor que é indicado por N XOR V, não apenas N. Como a ISA não possui branch direto em N XOR V, o software deve combinar testes quando valores com sinal e overflow forem possíveis.

> **JP:** JP testa apenas N=0. Ele significa 'não negativo' e também salta quando o resultado é zero; não equivale estritamente a 'maior que zero'.

8. Sistema de memória e inicialização

8.1 Mapa de memória de 64 KiB

Faixa	Tamanho	Seleção	Uso
0x0000–0x7FFF	32 KiB	/RAM_SEL / BIOS	RAM com o overlay liberado; ROM nos mesmos offsets com o overlay ativo.
0x8000–0x9FFF	8 KiB	/BIOS_SEL	ROM sempre visível; completa a janela ROM contínua de 40 KiB durante o overlay.
0xA000–0xEFFF	20 KiB	/CARD_SEL	Cartucho externo.
0xF000–0xFFFF	4 KiB	/IO_SEL	I/O mapeado em memória.

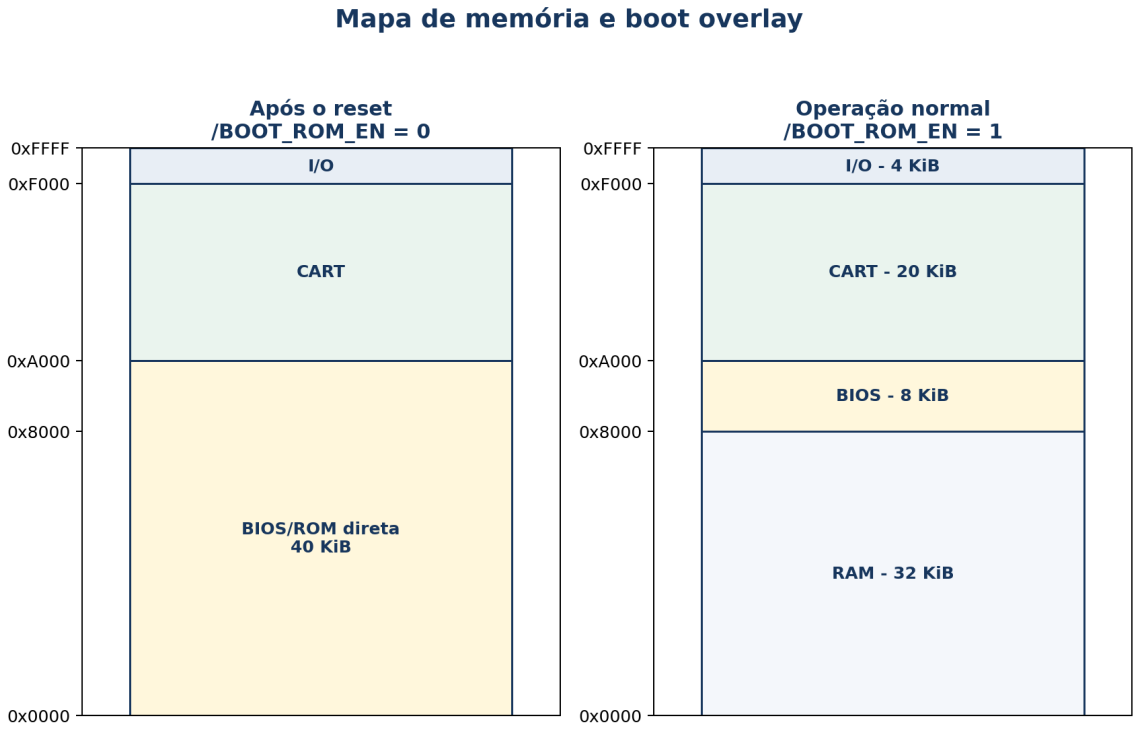


Figura 3 - Mapa de memória durante o reset e em operação normal.

8.2 RAM principal

A RAM é uma 62256 de 32 KiB x 8. A0-A14 recebem EXBUS[14:0], CE recebe /RAM_SEL, OE recebe /MEMORY_OE e WE recebe /MEMORY_WE. A RAM deve ser usada para variáveis, buffers, programas carregados e pilha.

O endereço recomendado para inicialização de SP é 0x7FFF. Como PUSH primeiro decrementa, o primeiro byte é gravado em 0x7FFE. Usar 0x8000 aproveitaria 0x7FFF no primeiro push, porém coloca SP temporariamente fora da RAM e não é a convenção documentada.

8.3 BIOS/ROM

A BIOS usa uma EPROM 27C512 de 64 KiB x 8. Todos os pinos A0-A15 estão ligados um a um ao External Address Bus, sem troca ou mascaramento de endereço: CPU 0x0000 lê o offset 0x0000 da

EPROM, CPU 0x8000 lê o offset 0x8000 e assim por diante. Os 24 KiB superiores da EPROM não ficam acessíveis neste mapa.

8.4 Boot overlay e transição para a RAM

Com /BOOT_ROM_EN=0, a EPROM ocupa de forma contínua os primeiros 40 KiB do espaço da CPU, de 0x0000 a 0x9FFF, e a RAM fica indisponível. O arquivo da ROM é lido normalmente nos mesmos offsets físicos, sem repetição automática. Esse modo pode ser mantido durante toda a execução quando o sistema preferir 40 KiB de ROM e não precisar de RAM.

Com /BOOT_ROM_EN=1, a RAM passa a ocupar 0x0000-0x7FFF e a EPROM continua visível apenas em 0x8000-0x9FFF. Para liberar a RAM, o código que ainda precisará executar depois da troca deve estar dentro dessa janela superior e a parte residente da BIOS deve caber em 8 KiB.

Offset na 27C512	Uso	Visibilidade
0x0000-0x7FFF	Programa/rotinas da ROM de boot	Visível somente enquanto o overlay permanece ativo.
0x8000-0x9FFF	Parte residente da BIOS	Visível tanto com o overlay ativo quanto com a RAM ativa.
0xA000-0xFFFF	Não mapeado para a BIOS	Não acessível pela CPU através da seleção da BIOS.

```
RESET: PC = 0x0000
  CPU executa diretamente EPROM[0x0000...]
  se for usar RAM: JMP 0x8000
  código residente em 0x8000 inicializa o hardware
  STI 0xF500, #0x01    ; RAM ativa, clock 16 MHz
  execução continua na janela ROM de 8 KiB
```

Transição segura: Não libere o overlay enquanto o PC executa abaixo de 0x8000, a menos que a RAM já contenha deliberadamente a continuação no próximo endereço. O fluxo normal salta primeiro para 0x8000-0x9FFF e só então escreve /BOOT_ROM_EN=1.

8.5 Região de cartucho

A janela 0xA000-0xEFFF totaliza 20 KiB e é selecionada por /CARD_SEL. O conector J1 fornece EXBUS[15:0], DBUS[7:0], /CARD_SEL, /CARD_IO_SEL, /MEMORY_OE, /MEMORY_WE, CLOCK_ADDMAR, +5 V, GND e /CARD_DETECT. /CARD_IO_SEL seleciona a página Y6 (0xF600-0xF6FF, espelhada em 0xFE00-0xFEFF). O módulo pode implementar ROM, EEPROM, Flash, SRAM, lógica ou periféricos próprios, desde que respeite o barramento e o decode do host.

/CARD_DETECT possui pull-up de 10 kohms na CPU. O cartucho indica presença aterrando o sinal. A detecção não torna o barramento hot-plug: inserir ou remover energizado não é suportado.

8.6 Aliases, limites e ausência de proteção

Não há MMU, privilégio, proteção contra escrita em I/O, proteção de pilha ou exceção de endereço. Um ponteiro pode atravessar RAM, ROM, cartucho e I/O; um SP incorreto pode escrever em periféricos. O software deve validar limites e reservar regiões de RAM por convenção.

Wraparound: PC, PTR e SP são contadores de 16 bits e podem retornar de 0xFFFF para 0x0000. INC/DEC PTR não atualizam flags, portanto o wrap não é sinalizado.

9. Arquitetura de I/O mapeado em memória

9.1 Decode por 74HC138

/IO_SEL cobre 0xF000-0xFFFF. Um 74HC138 usa A=A8, B=A9 e C=A10. As saídas Y0-Y7, ativas em nível baixo, selecionam páginas funcionais de 256 bytes. A11 não participa do sub-decode.

CBA	Saída	Página principal	Espelho	Módulo
000	Y0	0xF000–0xF0FF	0xF800–0xF8FF	Teclado
001	Y1	0xF100–0xF1FF	0xF900–0xF9FF	Display
010	Y2	0xF200–0xF2FF	0xFA00–0xFAFF	Vídeo/GPU
011	Y3	0xF300–0xF3FF	0xFB00–0xFBFF	Status/timer
100	Y4	0xF400–0xF4FF	0xFC00–0xFCFF	Som
101	Y5	0xF500–0xF5FF	0xFD00–0xFDFF	Configuração
110	Y6	0xF600–0xF6FF	0xFE00–0xFEFF	CART_IO
111	Y7	0xF700–0xF7FF	0xFF00–0xFFFF	Reservado

9.2 Espelhos e aliases

Como A11 é ignorado, cada página aparece novamente 0x800 bytes acima. Além disso, vários periféricos não decodificam A7-A1; por isso todos os endereços dentro da página podem acessar o mesmo registrador.

Bloco	Bits internos decodificados	Consequência
Teclado	Nenhum	Qualquer leitura na página equivale a KEY_DATA.
Display	A0	Endereços pares -> LOW; ímpares -> HIGH.
Status/timer	Nenhum	Qualquer leitura retorna status; qualquer escrita limpa os timers.
Som	Nenhum	Qualquer escrita na página atualiza SOUND_CFG.
Configuração	A0	Par -> CPU_CFG; ímpar -> PERIPH_CFG.
Vídeo	A7–A0	F200-F204 implementam DATA, CMD, X, Y e STATUS; os mesmos offsets aparecem em FA00-FA04.

9.3 Tipos de acesso e efeitos colaterais

Endereço	Acesso	Efeito
0xF000	R	Retorna 8 bits do teclado e assertará /KEY_RD durante a leitura.
0xF100/0xF101	W	Atualiza os latches de display; não há readback.

Endereço	Acesso	Efeito
0xF200	R/W	VIDEO_DATA: escrita dispara transferência; leitura retorna a última leitura de VRAM concluída.
0xF201	W	VIDEO_CMD: executa o comando escrito.
0xF202/0xF203	R/W	VIDEO_X/VIDEO_Y: posição da próxima operação.
0xF204	R	VIDEO_STATUS: BUSY, blanking, quadro e fim do cursor automático.
0xF300	R	Retorna timers e sinais de presença/ready.
0xF300	W	Dado ignorado; limpa simultaneamente TIMER_1MS e TIMER_500MS.
0xF400	W	Substitui todo SOUND_CFG; sem readback.
0xF500/0xF501	W	Substitui toda a configuração selecionada; sem readback.

10. Referência dos registradores de I/O

10.1 KEY_DATA - 0xF000

KEY_DATA é um registrador de leitura de 8 bits. U117 coloca KEYBOARD_B[7:0] no Data Bus quando /KEY_SEL e /MEMORY_OE estão ativos. A mesma condição gera /KEY_RD para o módulo de teclado; o periférico deve usar esse handshake para retirar o byte e limpar KEY_READY.

Bit	Nome	Significado
0	KEYBOARD_B0	Bit 0 do byte fornecido pelo teclado.
1	KEYBOARD_B1	Bit 1 do byte fornecido pelo teclado.
2	KEYBOARD_B2	Bit 2 do byte fornecido pelo teclado.
3	KEYBOARD_B3	Bit 3 do byte fornecido pelo teclado.
4	KEYBOARD_B4	Bit 4 do byte fornecido pelo teclado.
5	KEYBOARD_B5	Bit 5 do byte fornecido pelo teclado.
6	KEYBOARD_B6	Bit 6 do byte fornecido pelo teclado.
7	KEYBOARD_B7	Bit 7 do byte fornecido pelo teclado.

> **Sequência:** Teste primeiro /KEYBOARD_DETECT no status; depois KEY_READY; somente então leia 0xF000. Ler sem byte pronto pode reconhecer ou descartar um valor inválido dependendo da implementação do teclado.

10.2 DISPLAY_LOW/HIGH - 0xF100/0xF101

Dois 74HC377 armazenam os bytes inferior e superior do valor exibido. Duas EPROMs 27C512 convertem cada byte em segmentos para um par de dígitos; DISP_CLOCK seleciona alternadamente os nibbles. O gerador 7Seg(1).py define de forma normativa os modos HEX e CHAR.

Endereço	Latch	Dígitos lógicos
0xF100	DISPLAY_LOW	bits 3:0 -> dígito 0; bits 7:4 -> dígito 1
0xF101	DISPLAY_HIGH	bits 3:0 -> dígito 2; bits 7:4 -> dígito 3

Endereço da 27C512	Origem	Função
A0-A7	Byte do latch	Valor de entrada a decodificar.
A8 = 0 / 1	DISP_CLOCK	0 seleciona nibble baixo/dígito direito; 1 seleciona nibble alto/dígito esquerdo.
A9 = 0 / 1	/DISP_HEX	0 seleciona HEX; 1 seleciona CHAR.
A10-A15	GND	Mantidos em zero.

Cada imagem tem 65.536 bytes. display_low.bin e display_high.bin são idênticos; o que muda é o byte ligado a A0-A7 de cada EPROM. D0-D6 dirigem, respectivamente, os segmentos A-G de displays de cátodo comum; D7 é o ponto decimal e não é usado.

Código CHAR	Símbolo	Código CHAR	Símbolo
0	espaço	8	L
1	A	9	O
2	b	A	P
3	C	B	r
4	d	C	t
5	E	D	U
6	F	E	-
7	H	F	_

O multiplex usa um 555, dois BC337, resistores de base de 1 kohm e resistores de 330 ohms por segmento.

10.3 VIDEO_BASE - 0xF200

A página de vídeo é encaminhada ao conector J4. O contrato CPU-visível usa cinco offsets em 0xF200-0xF204, repetidos em 0xFA00-0xFA04 por causa do espelho da página Y2.

Endereço	Nome	Acesso	Contrato com a CPU
0xF200	VIDEO_DATA	R/W	Escrita retém o byte e dispara uma operação pendente de VRAM. Leitura retorna o resultado da última leitura concluída.
0xF201	VIDEO_CMD	W	Recebe um código de comando completo.
0xF202	VIDEO_X	R/W	Posição horizontal: coluna de caractere no texto ou coluna de byte nos modos gráficos.
0xF203	VIDEO_Y	R/W	Linha da operação. Escrever X ou Y também limpa CURSOR_END.
0xF204	VIDEO_STATUS	R	Estado da interface e da varredura.

Comando	Nome	Função
0x00	VIDEO_CLEAR	Limpa a área usada pelo modo atual.
0x04	CURSOR_HIDE	Esconde o cursor no modo texto.
0x05	CURSOR_SHOW	Mostra o cursor no modo texto.
0x11	MODE_TEXT_80X30	Texto 80 x 30, fonte 8 x 8 e escala 2x apenas na vertical.
0x12	MODE_GFX_320X240	Gráfico 320 x 240 monocromático, escala 2x; 40 bytes por linha.
0x13	MODE_GFX_160X120	Gráfico 160 x 120 monocromático, escala 4x; 20 bytes por linha.
0x20	VRAM_READ	Solicita leitura da VRAM na posição X/Y; não incrementa a posição.

Modo	VIDEO_X	VIDEO_Y	Bytes/linha	VIDEO_DATA
Texto 80 x 30	0–79	0–29	80	Código de caractere.
Gráfico 320 x 240	0–39	0–239	40	Oito pixels; bit 7 é o pixel esquerdo.
Gráfico 160 x 120	0–19	0–119	20	Oito pixels; bit 7 é o pixel esquerdo.

Bit	VIDEO_STATUS	Significado
0	BUSY	1 = X, Y, DATA e a operação atual estão reservados; aguarde antes de nova transação.
1	VBLANK	1 durante o intervalo vertical.
2	HBLANK	1 durante o intervalo horizontal.
3	FRAME	Alterna entre 0 e 1 a cada quadro completo.
4	CURSOR_END	Flag retida: a última posição válida foi escrita e o cursor automático permaneceu nela.
5–7	RES0	Reservados; leitura deve retornar zero.

> **Handshake obrigatório:** Enquanto BUSY=1, o software pode ler VIDEO_STATUS, mas não deve alterar VIDEO_X, VIDEO_Y ou VIDEO_DATA nem iniciar um novo comando que use a VRAM. Uma escrita em VIDEO_DATA incrementa X/Y somente depois da gravação física e antes de BUSY voltar a zero.

```
VIDEO_WAIT:
    LDA $F204
    TEST A, #$01
    JNZ VIDEO_WAIT
    RET

; Exemplo: seleciona texto, posiciona e envia 'A'
CALL VIDEO_WAIT
STI $F201, #$11
CALL VIDEO_WAIT
STI $F202, #$00
STI $F203, #$00
STI $F200, #$41
```

SYSTEM_STATUS bit 5 fornece /VIDEO_DETECT: 0 indica placa presente e 1 indica ausência. Resoluções, temporização de vídeo, arbitragem interna, VRAM e implementação elétrica detalhada pertencem ao manual específico da GPU; aqui permanece somente a interface necessária ao software do WALL-8.

10.4 SYSTEM_STATUS - 0xF300

Bit	Nome	Leitura
0	TIMER_1MS	1 quando ocorreu ao menos um tick desde a última limpeza.
1	TIMER_500MS	1 quando ocorreu ao menos um tick desde a última limpeza.
2	RES0	Reservado; ligado em nível baixo.
3	RES0	Reservado; ligado em nível baixo.
4	/CARD_DETECT	0 = cartucho conectado; 1 = ausente.
5	/VIDEO_DETECT	0 = vídeo conectado; 1 = ausente.
6	/KEYBOARD_DETECT	0 = teclado conectado; 1 = ausente.

Bit	Nome	Leitura
7	KEY_READY	1 = existe byte aguardando leitura.

Uma escrita de qualquer valor em qualquer alias de TIMER_SEL limpa os dois flip-flops de timer. Sinais de presença e KEY_READY não são alterados pela escrita.

> **Coalescência:** Os timers são flags de evento. Dez ticks ocorridos antes da leitura continuam aparecendo como um único bit 1. Ao limpar TIMER_1MS, TIMER_500MS também é limpo; leia e processe o snapshot completo antes de escrever.

10.5 SOUND_CFG - 0xF400

Bit	Nome	Função
0	NOISE_EN	0 = gerador em reset/desligado; 1 = habilitado.
1	TONE_EN	0 = gerador em reset/desligado; 1 = habilitado.
2	TONE_B0	LSB da seleção do resistor/frequência de tom.
3	TONE_B1	Bit 1 da seleção de tom.
4	TONE_B2	MSB da seleção de tom.
5	NOISE_B0	LSB da seleção/configuração de ruído.
6	NOISE_B1	Bit 1 da seleção de ruído.
7	NOISE_B2	MSB da seleção de ruído.

O tom e o clock do ruído são gerados por um NE556. Dois 74HC4051 selecionam um entre oito ajustes resistivos. O ruído usa um 74HC164 e realimentação XOR. Os sinais são somados, passam por RV3 e alimentam o amplificador TDA2822M em ponte; o ponto AUDIO_MIX também alimenta a saída P2.

10.6 CPU_CFG - 0xF500

Bit	Nome	Reset	Função
0	/BOOT_ROM_EN	0	0 = EPROM selecionada na região baixa; 1 = RAM selecionada.
1	RES	0	Reservado; escrever 0.
2	RES	0	Reservado; escrever 0.
3	RES	0	Reservado; escrever 0.
4	CLK_B0	0	LSB da seleção de clock.
5	CLK_B1	0	Bit 1 da seleção de clock.
6	CLK_B2	0	MSB da seleção de clock.
7	RES	0	Reservado; escrever 0.

CPU_CFG é write-only e cada escrita substitui todos os oito bits. O software deve manter uma variável shadow caso altere o clock depois do boot. Não use read-modify-write em 0xF500.

10.7 PERIPH_CFG - 0xF501

Bit	Nome	Reset	Função
0	/DISP_HEX	0	0 = modo HEX; 1 = modo CHAR definido por 7Seg(1).py.
1	RES	0	Reservado; escrever 0.
2	/LED_EN	0	0 = LEDs de diagnóstico habilitados; 1 = desabilitados.
3	RES	0	Reservado; escrever 0.
4	RES	0	Reservado; escrever 0.
5	RES	0	Reservado; escrever 0.
6	RES	0	Reservado; escrever 0.
7	RES	0	Reservado; escrever 0.

PERIPH_CFG também é write-only. No modo CHAR, cada nibble usa a tabela espaço/A/b/C/d/E/F/H/L/O/P/r/t/U/-/_ apresentada na Seção 10.2.

10.8 CART_IO - Y6 e Y7 reservado

Y6 (0xF600-0xF6FF, espelhada em 0xFE00-0xFEFF) é a região CART_IO e é encaminhada ao conector J1 por /CARD_IO_SEL. O WALL-8 não define registradores nessa página: a decodificação dos offsets e suas funções são definidas pelo hardware de cada cartucho e controladas pelo código da própria ROM do cartucho. Y7 (0xF700-0xF7FF, espelhada em 0xFF00-0xFFFF) permanece reservado para expansão futura.

11. Interfaces externas e pinagem

11.1 Teclado - J2

J2 é um conector de 16 vias. O teclado fornece um byte paralelo, KEY_READY e /KEYBOARD_DETECT; a CPU fornece +5 V, GND e /KEY_RD. /KEY_RD permanece alto por pull-up e é assertado em nível baixo durante uma leitura de KEY_DATA.

Pino ímpar	Sinal	Pino par	Sinal
1	KEYBOARD_B0	2	+5 V
3	KEYBOARD_B1	4	NC
5	KEYBOARD_B2	6	NC
7	KEYBOARD_B3	8	/KEY_RD
9	KEYBOARD_B4	10	/KEYBOARD_DETECT
11	KEYBOARD_B5	12	KEY_READY
13	KEYBOARD_B6	14	NC
15	KEYBOARD_B7	16	GND

- /KEYBOARD_DETECT deve ser levado a GND pelo módulo presente; R181 de 10 kohms o mantém em 1 quando ausente.
- KEY_READY deve ir a 1 quando o byte estiver estável e retornar a 0 após /KEY_RD, conforme a lógica do teclado.
- KEY_READY e KEYBOARD_B0-B6 possuem pull-downs de 10 kohms na folha atual; não dependa do byte de dados quando o módulo estiver ausente.

11.2 Cartucho - J1

J1 é um conector 2x20. A pinagem distribui alimentação nos extremos, intercala endereço/dados e concentra os controles na região central.

Pino	Sinal	Pino	Sinal
1	GND	2	+5 V
3	GND	4	+5 V
5	EXBUS0	6	DBUS0
7	EXBUS1	8	DBUS1
9	EXBUS2	10	DBUS2
11	EXBUS3	12	DBUS3
13	EXBUS4	14	DBUS4
15	EXBUS5	16	DBUS5
17	EXBUS6	18	DBUS6
19	EXBUS7	20	DBUS7

Pino	Sinal	Pino	Sinal
21	EXBUS8	22	CLOCK_ADDMAR
23	EXBUS9	24	/CARD_SEL
25	EXBUS10	26	/MEMORY_OE
27	EXBUS11	28	/MEMORY_WE
29	EXBUS12	30	/CARD_DETECT
31	EXBUS13	32	NC
33	EXBUS14	34	/CARD_IO_SEL
35	EXBUS15	36	NC
37	GND	38	+5 V
39	GND	40	+5 V

Clock do cartucho: CLOCK_ADDMAR é um ramo do clock distribuído. O módulo não deve assumir que /CARD_SEL ou /CARD_IO_SEL permanecerão ativos em todos os seus flancos. Um acesso à memória do cartucho é definido por endereço, /CARD_SEL e /MEMORY_OE ou /MEMORY_WE; um acesso ao I/O do cartucho é definido por endereço, /CARD_IO_SEL e /MEMORY_OE ou /MEMORY_WE.

11.3 Vídeo/GPU - J4

J4 usa o mesmo corpo 2x20, mas posiciona CLOCK_PeRA no pino 34 e /VIDEO_SEL no pino 24.

Pino	Sinal	Pino	Sinal
1	GND	2	+5 V
3	GND	4	+5 V
5	EXBUS0	6	DBUS0
7	EXBUS1	8	DBUS1
9	EXBUS2	10	DBUS2
11	EXBUS3	12	DBUS3
13	EXBUS4	14	DBUS4
15	EXBUS5	16	DBUS5
17	EXBUS6	18	DBUS6
19	EXBUS7	20	DBUS7
21	EXBUS8	22	NC
23	EXBUS9	24	/VIDEO_SEL
25	EXBUS10	26	/MEMORY_OE
27	EXBUS11	28	/MEMORY_WE
29	EXBUS12	30	/VIDEO_DETECT
31	EXBUS13	32	NC
33	EXBUS14	34	CLOCK_PeRA
35	EXBUS15	36	NC

Pino	Sinal	Pino	Sinal
37	GND	38	+5 V
39	GND	40	+5 V

R180 de 10 kohms mantém /VIDEO_DETECT em nível alto. O módulo presente deve aterramento lógico desse sinal. Como os 16 bits de endereço são expostos, a GPU pode distinguir os aliases se desejar, mas o host considera 0xF2xx e 0xFAxx a mesma seleção Y2.

11.4 Alimentação e áudio - J5, J3 e J6

Conector	Tipo	Função / cuidado
J5	Terminal 2 vias	Entrada +5 V/GND; pino 2 passa por FU1 de 3 A.
J3	Speaker 2 vias	Saída em ponte do TDA2822M. Nenhum terminal do alto-falante deve ser aterrado.
J6	P2 estéreo PJ-307-5	AUDIO_MIX mono AC-coplado e duplicado para tip/ring por resistores de 1 kohm.

> **Saída bridge:** J3 é saída de potência diferencial. Não conecte um dos pinos do alto-falante ao GND nem ligue essa saída diretamente a uma entrada de áudio aterrada. Para line/headphone use J6, que deriva do ponto AUDIO_MIX anterior ao estágio bridge.

A saída P2 é passiva e não possui especificação de potência para fones de baixa impedância. O nível final deve ser caracterizado; trate-a inicialmente como saída mono de linha/monitoramento.

11.5 Regras de integração de módulos

1. Com a placa desligada, confirme GND, +5 V e orientação do conector.
2. Garanta que toda saída ligada ao Data Bus entre em alta impedância quando /SEL ou /MEMORY_OE não estiverem ativos.
3. Use /DETECT apenas como indicação de presença; ele não autoriza hot-plug.
4. Valide tempos de propagação no clock escolhido. Se o módulo não cumprir 16 MHz, selecione um divisor mais lento.
5. Para MCUs de 3,3 V, use level shifters apropriados e buffers tri-state; não confie apenas em valores típicos.
6. Não ultrapasse a capacidade da fonte e do conector ao alimentar GPU/cartucho pelo +5 V da CPU.

12. Unidade de controle e microcódigo

12.1 Formato do endereço da micro-ROM

As cinco ROMs recebem o mesmo endereço de 16 bits. O gerador usa a expressão abaixo; opcode ocupa os bits menos significativos, flags vêm em seguida e o Step Counter ocupa os bits mais altos.

```
micro_addr = opcode | (flags << 8) | (step << 12)
flags = (Z << 0) | (C << 1) | (N << 2) | (V << 3)
```

Endereço da ROM	Origem	Ordem
A0-A7	IR_BIT_0..7	Opcode bit 0 no A0.
A8	Z	Flag bit 0.
A9	C	Flag bit 1.
A10	N	Flag bit 2.
A11	V	Flag bit 3.
A12-A15	STEP_BIT_0..3	T0=0000 até T15=1111.

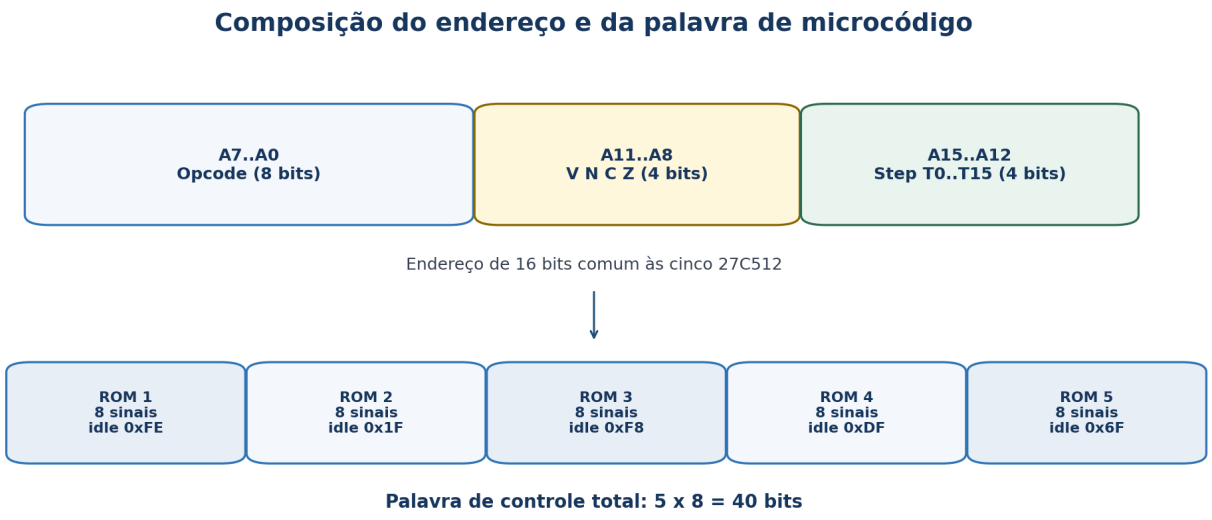


Figura 4 - Endereço comum e palavra de controle das cinco micro-ROMs.

12.2 Palavra de controle de 40 bits

Cada 27C512 fornece oito saídas. Em conjunto elas formam uma microinstrução de 40 bits. A tabela resume a distribuição; o Apêndice B descreve cada linha individualmente.

ROM	D0	D1	D2	D3	D4	D5	D6	D7
1	PC_INC	/PC_LOAD	/PC_OUT	/MAR_LOAD	/MAR_OUT	/IR_LOAD	/RA_LOAD	/RA_OUT
2	/RB_LOAD	/RB_OUT	/ ADDR_L_LOAD	/ADDR_H_LOAD	/ADDR_OUT	AB_SWAP	NOT_X	SUB
3	MUX0	MUX1	MUX2	/ALU_OUT	/MEM_OE	/MEM_WE	/FLAGS_LOAD	/STEP_RESET
4	/HALT	/PTR_EN	/PTR_L_LOAD	/PTR_H_LOAD	/PTR_OUT	PTR_DOWN	/TMP_LOAD	/TMP_OUT
5	/SP_EN	/SP_L_LOAD	/SP_H_LOAD	/SP_OUT	SP_DOWN	/PC_L_DATA	/PC_H_DATA	NC

12.3 Valores neutros e polaridade

O gerador inicializa todos os 65.536 endereços de cada ROM com um byte neutro. Depois, set_step altera apenas os bits necessários. Sinais com polaridade H são ativados escrevendo 1; sinais L são ativados escrevendo 0.

ROM	Idle	Binário	Razão
1	0xFE	11111110	PC_INC baixo; sete controles ativos-baixo em 1.
2	0x1F	00011111	Cinco controles ativos-baixo em 1; três controles ALU em 0.
3	0xF8	11111000	MUX=000; cinco controles ativos-baixo em 1.
4	0xDF	11011111	PTR_DOWN=0 e demais controles desabilitados.
5	0x6F	01101111	SP_DOWN=0; D7 é NC e seu nível não tem função.

Exemplo: T0 do fetch ativa PC_OUT e MAR_LOAD, resultando em ROM1=0xF2 e as demais ROMs em idle. T1 ativa PC_INC, MAR_OUT, IR_LOAD e MEMORY_OE, resultando em ROM1=0xCF e ROM3=0xE8.

12.4 Codificação das funções da ALU

ALU_MUX_2..0 seleciona um de oito caminhos. ALU_SUB e ALU_NOT_X refinam as funções aritméticas ou XOR/NOT. A codificação completa está na Seção 7.2 e é reproduzida no gerador pelas listas ALU_ADD_OP, ALU_SUB_OP, ALU_AND_OP, ALU_OR_OP, ALU_XOR_OP, ALU_NOT_OP, ALU_SHR_OP, ALU_SHL_OP, ALU_INC_OP, ALU_DEC_OP e ALU_ZERO_OP.

12.5 Fetch comum e encerramento da instrução

```
T0: PC_OUT + MAR_LOAD
T1: MAR_OUT + MEMORY_OE + IR_LOAD + PC_INC
```

O gerador grava T0/T1 para todos os 256 opcodes. A instrução específica começa em T2. O último microstep normalmente assertará STEP_RESET; no próximo flanco o contador volta a T0 e um novo fetch é iniciado.

Jumps condicionais sempre leem os dois bytes do destino. Em T6, PC_LOAD só é ativado nas oito combinações de flags que atendem à condição; T7 encerra a instrução em ambos os casos.

12.6 Geração e gravação das ROMs

7. Edite Gerador_Microcodigo.py preservando o endereço opcode/flags/step.
8. Declare sinais como tuplas (rom, bit, polaridade) e use set_step para cada T-state.
9. Execute o script; ele gera microcode_rom1.bin a microcode_rom5.bin.
10. Confirme que cada arquivo possui exatamente 65.536 bytes.
11. Compare os checksums com a baseline esperada ou registre uma nova revisão.
12. Grave uma 27C512 para cada posição ROM1-ROM5, sem trocar a ordem.
13. Teste em clock manual antes de habilitar o clock principal.

> **Capacidade:** As cinco imagens totalizam 327.680 bytes (320 KiB), embora cada microinstrução efetiva possua 40 bits. Flags multiplicam cada opcode por 16 páginas e o Step Counter por mais 16 posições.

12.7 Opcodes não definidos

Como T0/T1 são escritos para todos os opcodes e os demais endereços ficam neutros, um opcode não definido executa o fetch, permanece sem efeitos de T2 a T15 e volta naturalmente a T0 quando o contador de 4 bits transborda. O comportamento atual equivale a um NOP de 16 ciclos e um byte.

> **Não depender:** Esse comportamento não é uma exceção arquitetural garantida. Uma futura ROM pode transformar opcodes ilegais em HALT, trap ou instruções novas. Software válido deve usar somente os 76 opcodes definidos.

13. Codificação e convenções da ISA

13.1 Sintaxe do customasm

O arquivo wall8.cpu usa #ruledef do customasm. Um programa inclui a definição e escreve os mnemônicos exatamente conforme as regras. O cifrão marca números hexadecimais; # identifica imediato.

Agradecimento: o projeto WALL-8 utiliza o montador customasm para definir e montar sua ISA. Obrigado aos autores e colaboradores do projeto: repositório oficial do customasm

```
#include "wall8.cpu"

SP $7FFF
LDA #$55
STA $0010
HLT
```

Forma	Significado
A / B	Registradores de 8 bits.
#$xx	Imediato de 8 bits.
$addr	Endereço absoluto de 16 bits.
[PTR]	Byte em MEM[PTR].
[PTR+]	Byte em MEM[PTR], seguido de PTR <- PTR+1.
label	Símbolo resolvido pelo montador para um endereço absoluto.

13.2 Formatos de instrução

Formato	Bytes	Exemplo	Codificação
Implícito	1	INC A	42
Imediato 8	2	LDA #$55	10 55
Absoluto 16	3	LDA $1234	12 34 12
Store imediato	4	STI $1234, #$55	27 34 12 55
Indireto PTR	1	LDA [PTR+]	23

13.3 Little-endian e cálculo do PC

Todo endereço absoluto é codificado como LL HH. PC aponta para o opcode antes do fetch; T1 carrega IR e incrementa PC. Cada byte de operando absoluto é lido em um par de microsteps e incrementa PC. Depois de CALL ler LL/HH, PC já contém o endereço da instrução seguinte - exatamente o valor empilhado como retorno.

13.4 Ciclos e flags

A contagem de ciclos inclui T0, T1 e o microstep final que executa STEP_RESET. HLT consome T0/T1 e para no T2. Loads, stores, movimentos, jumps e pilha não atualizam flags. Todas as operações da ALU listadas como SIM gravam Z, C, N e V juntas; não há atualização parcial.

Classe	Ciclos típicos
NOP / HLT	3
ALU registrador / MOV / INC PTR	4
Load imediato / load [PTR]	5
Store [PTR] / POP	6
PTR/SP absoluto / PUSH / PTR+ store	7
Jump / ALU imediato	8
Load absoluto	9
Store absoluto / RET	10
STI	12
CALL	16

14. Referência completa das 76 instruções

As tabelas seguintes são geradas a partir da referência de instruções e conferidas contra wall8.cpu. 'Flags' indica uma nova gravação de Z/C/N/V; 'A/B' considera o estado final visível, não os usos temporários de TMP.

14.1 Sistema e controle

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x00	NOP	00	3	NÃO	NÃO	Não executa operação e inicia o próximo fetch. Ex.: NOP
0x01	HLT	01	3	NÃO	NÃO	Para a CPU até ocorrer reset. Ex.: HLT

14.2 Load, store e movimentação

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x10	LDA #$xx	10 xx	5	NÃO	SIM	Carrega um valor imediato de 8 bits em A. Ex.: LDA #$55
0x11	LDB #$xx	11 xx	5	NÃO	SIM	Carrega um valor imediato de 8 bits em B. Ex.: LDB #$0A
0x12	LDA $addr	12 LL HH	9	NÃO	SIM	Lê a memória no endereço absoluto e grava em A. Ex.: LDA $F300
0x13	LDB $addr	13 LL HH	9	NÃO	SIM	Lê a memória no endereço absoluto e grava em B. Ex.: LDB $2000
0x14	STA $addr	14 LL HH	10	NÃO	NÃO	Grava A no endereço absoluto. Ex.: STA $F100
0x15	STB $addr	15 LL HH	10	NÃO	NÃO	Grava B no endereço absoluto. Ex.: STB $0100
0x16	MOV A, B	16	4	NÃO	SIM	Copia o valor de B para A. Ex.: MOV A, B
0x17	MOV B, A	17	4	NÃO	SIM	Copia o valor de A para B. Ex.: MOV B, A
0x18	LDA [PTR]	18	5	NÃO	SIM	Lê o byte apontado por PTR e grava em A. Ex.: PTR $8000 / LDA [PTR]
0x19	LDB [PTR]	19	5	NÃO	SIM	Lê o byte apontado por PTR e grava em B. Ex.: PTR $8000 / LDB [PTR]
0x1A	STA [PTR]	1A	6	NÃO	NÃO	Grava A no endereço apontado por PTR. Ex.: PTR $0200 / STA [PTR]
0x1B	STB [PTR]	1B	6	NÃO	NÃO	Grava B no endereço apontado por PTR. Ex.: PTR $0200 / STB [PTR]
0x27	STI $addr, #$xx	27 LL HH xx	12	NÃO	NÃO	Grava um valor imediato diretamente na memória. Ex.: STI $F400, #$12

14.3 Ponteiro PTR

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x20	PTR $addr	20 LL HH	7	NÃO	NÃO	Carrega um endereço de 16 bits em PTR. Ex.: PTR $8000
0x21	INC PTR	21	4	NÃO	NÃO	Incrementa PTR em 1. Ex.: INC PTR
0x22	DEC PTR	22	4	NÃO	NÃO	Decrementa PTR em 1. Ex.: DEC PTR
0x23	LDA [PTR+]	23	6	NÃO	SIM	Lê MEM[PTR] para A e depois incrementa PTR. Ex.: LDA [PTR+]
0x24	LDB [PTR+]	24	6	NÃO	SIM	Lê MEM[PTR] para B e depois incrementa PTR. Ex.: LDB [PTR+]
0x25	STA [PTR+]	25	7	NÃO	NÃO	Grava A em MEM[PTR] e depois incrementa PTR. Ex.: STA [PTR+]
0x26	STB [PTR+]	26	7	NÃO	NÃO	Grava B em MEM[PTR] e depois incrementa PTR. Ex.: STB [PTR+]

14.4 ALU entre registradores

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x30	ADD A, B	30	4	SIM	SIM	Soma B a A e grava o resultado em A. Ex.: ADD A, B
0x31	ADD B, A	31	4	SIM	SIM	Soma A a B e grava o resultado em B. Ex.: ADD B, A
0x32	SUB A, B	32	4	SIM	SIM	Subtrai B de A e grava o resultado em A. Ex.: SUB A, B
0x33	SUB B, A	33	4	SIM	SIM	Subtrai A de B e grava o resultado em B. Ex.: SUB B, A
0x34	AND A, B	34	4	SIM	SIM	Executa A AND B e grava o resultado em A. Ex.: AND A, B
0x35	AND B, A	35	4	SIM	SIM	Executa B AND A e grava o resultado em B. Ex.: AND B, A
0x36	OR A, B	36	4	SIM	SIM	Executa A OR B e grava o resultado em A. Ex.: OR A, B
0x37	OR B, A	37	4	SIM	SIM	Executa B OR A e grava o resultado em B. Ex.: OR B, A
0x38	XOR A, B	38	4	SIM	SIM	Executa A XOR B e grava o resultado em A. Ex.: XOR A, B
0x39	XOR B, A	39	4	SIM	SIM	Executa B XOR A e grava o resultado em B. Ex.: XOR B, A
0x3A	NOT A	3A	4	SIM	SIM	Inverte todos os bits de A. Ex.: NOT A
0x3B	NOT B	3B	4	SIM	SIM	Inverte todos os bits de B. Ex.: NOT B
0x3C	SHL A	3C	4	SIM	SIM	Desloca A um bit para a esquerda. Ex.: SHL A
0x3D	SHR A	3D	4	SIM	SIM	Desloca A um bit para a direita. Ex.: SHR A
0x3E	SHL B	3E	4	SIM	SIM	Desloca B um bit para a esquerda. Ex.: SHL B

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x3F	SHR B	3F	4	SIM	SIM	Desloca B um bit para a direita. Ex.: SHR B

14.5 Comparação, incremento e clear

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x40	CMP A, B	40	4	SIM	NÃO	Compara A com B usando o resultado de A − B. Ex.: CMP A, B / JZ IGUAIS
0x41	CMP B, A	41	4	SIM	NÃO	Compara B com A usando o resultado de B − A. Ex.: CMP B, A / JN MENOR
0x42	INC A	42	4	SIM	SIM	Incrementa A em 1. Ex.: INC A
0x43	DEC A	43	4	SIM	SIM	Decrementa A em 1. Ex.: DEC A
0x44	INC B	44	4	SIM	SIM	Incrementa B em 1. Ex.: INC B
0x45	DEC B	45	4	SIM	SIM	Decrementa B em 1. Ex.: DEC B
0x46	CLR A	46	4	SIM	SIM	Carrega 0x00 em A. Ex.: CLR A
0x47	CLR B	47	4	SIM	SIM	Carrega 0x00 em B. Ex.: CLR B

14.6 Jumps, pilha e subrotinas

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x50	JMP $addr	50 LL HH	8	NÃO	NÃO	Salta incondicionalmente para o endereço. Ex.: JMP LOOP
0x51	JZ $addr	51 LL HH	8	NÃO	NÃO	Salta quando Z = 1. Ex.: CMP A, #$00 / JZ ZERO
0x52	JNZ $addr	52 LL HH	8	NÃO	NÃO	Salta quando Z = 0. Ex.: TEST A, #$80 / JNZ BIT_LIGADO
0x53	JC $addr	53 LL HH	8	NÃO	NÃO	Salta quando C = 1. Ex.: ADD A, B / JC CARRY
0x54	JNC $addr	54 LL HH	8	NÃO	NÃO	Salta quando C = 0. Ex.: ADD A, B / JNC SEM_CARRY
0x55	JN $addr	55 LL HH	8	NÃO	NÃO	Salta quando N = 1. Ex.: CMP A, B / JN NEGATIVO
0x56	JP $addr	56 LL HH	8	NÃO	NÃO	Salta quando N = 0. Ex.: CMP A, B / JP NAO_NEGATIVO
0x57	JV $addr	57 LL HH	8	NÃO	NÃO	Salta quando V = 1. Ex.: ADD A, B / JV OVERFLOW
0x58	JNV $addr	58 LL HH	8	NÃO	NÃO	Salta quando V = 0. Ex.: ADD A, B / JNV SEM_OVERFLOW
0x59	CALL $addr	59 LL HH	16	NÃO	NÃO	Empilha o endereço de retorno e chama uma subrotina. Ex.: CALL DESENHA_TELA
0x5A	RET	5A	10	NÃO	NÃO	Retorna ao endereço salvo pelo CALL. Ex.: MINHA_ROTINA: / INC A / RET

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x5B	PUSH A	5B	7	NÃO	NÃO	Salva A na pilha. Ex.: PUSH A
0x5C	PUSH B	5C	7	NÃO	NÃO	Salva B na pilha. Ex.: PUSH B
0x5D	POP A	5D	6	NÃO	SIM	Recupera da pilha um valor para A. Ex.: POP A
0x5E	POP B	5E	6	NÃO	SIM	Recupera da pilha um valor para B. Ex.: POP B
0x5F	SP $addr	5F LL HH	7	NÃO	NÃO	Inicializa o Stack Pointer com um endereço de 16 bits. Ex.: SP $7FFF

14.7 ALU com valor imediato

Opcode	Assembly	Bytes	Ciclos	Flags	A/B	Efeito e exemplo
0x60	ADD A, #$xx	60 xx	8	SIM	SIM	Soma o valor imediato a A. Ex.: ADD A, #$05
0x61	ADD B, #$xx	61 xx	8	SIM	SIM	Soma o valor imediato a B. Ex.: ADD B, #$05
0x62	SUB A, #$xx	62 xx	8	SIM	SIM	Subtrai o valor imediato de A. Ex.: SUB A, #$01
0x63	SUB B, #$xx	63 xx	8	SIM	SIM	Subtrai o valor imediato de B. Ex.: SUB B, #$01
0x64	AND A, #$xx	64 xx	8	SIM	SIM	Executa A AND valor imediato e grava o resultado em A. Ex.: AND A, #$0F
0x65	AND B, #$xx	65 xx	8	SIM	SIM	Executa B AND valor imediato e grava o resultado em B. Ex.: AND B, #$F0
0x66	OR A, #$xx	66 xx	8	SIM	SIM	Executa A OR valor imediato e grava o resultado em A. Ex.: OR A, #$80
0x67	OR B, #$xx	67 xx	8	SIM	SIM	Executa B OR valor imediato e grava o resultado em B. Ex.: OR B, #$01
0x68	XOR A, #$xx	68 xx	8	SIM	SIM	Executa A XOR valor imediato e grava o resultado em A. Ex.: XOR A, #$FF
0x69	XOR B, #$xx	69 xx	8	SIM	SIM	Executa B XOR valor imediato e grava o resultado em B. Ex.: XOR B, #$FF
0x6A	CMP A, #$xx	6A xx	8	SIM	NÃO	Compara A com o valor imediato usando A − imediato. Ex.: CMP A, #$FF / JZ IGUAL
0x6B	CMP B, #$xx	6B xx	8	SIM	NÃO	Compara B com o valor imediato usando B − imediato. Ex.: CMP B, #$10 / JNZ DIFERENTE
0x6C	TEST A, #$xx	6C xx	8	SIM	NÃO	Testa em A os bits selecionados pela máscara imediata. Ex.: TEST A, #$80 / JNZ BIT_7_LIGADO
0x6D	TEST B, #$xx	6D xx	8	SIM	NÃO	Testa em B os bits selecionados pela máscara imediata. Ex.: TEST B, #$10 / JZ BIT_4_DESLIGADO

15. Pilha, subrotinas e práticas de programação

15.1 Organização da pilha

A pilha cresce para endereços menores. SP aponta para o primeiro item válido após um push; quando vazia, contém o limite superior escolhido pelo software. A baseline recomenda SP $7FFF, fazendo o primeiro push usar 0x7FFE.

```
PUSH: SP <- SP - 1; MEM[SP] <- valor
POP:  valor <- MEM[SP]; SP <- SP + 1
```

Sem proteção: PUSH em excesso pode atravessar dados, código em RAM e I/O. POP com pilha vazia lê o endereço apontado sem gerar exceção.

15.2 CALL, RET, PUSH e POP

CALL lê o destino, salva o PC já avançado e então carrega o destino. A ordem de bytes na memória foi escolhida para que RET leia low primeiro.

Fase	CALL	RET
1	Lê LL/HH do destino; PC aponta para retorno.	MAR <- SP; ADDR_L <- MEM[SP].
2	SP--; grava PC high.	SP++; MAR <- SP; ADDR_H <- MEM[SP].
3	SP--; grava PC low.	SP++; PC <- ADDR.
4	PC <- destino.	Encerra.

CALL/RET não salvam flags, A, B ou PTR. Uma subrotina deve preservar explicitamente os registradores exigidos pela convenção do programa.

15.3 Convenção de chamada recomendada

Recurso	Convenção sugerida
A	Primeiro argumento e valor de retorno; caller-saved.
B	Segundo argumento/temporário; caller-saved, salvo quando necessário.
PTR	Ponteiro de trabalho; callee deve documentar se o destrói.
Flags	Sempre caller-saved de forma lógica: qualquer ALU pode substituí-las e não há PUSH FLAGS.
SP	Deve sair da rotina com o mesmo valor de entrada.

Essa convenção não é implementada pelo hardware; serve como ABI inicial para BIOS e programas. Quando houver compilador ou monitor, a ABI deverá receber uma revisão própria.

15.4 Shadow de registradores write-only

SOUND_CFG, CPU_CFG e PERIPH_CFG não possuem caminho de readback. Para alterar apenas um campo, mantenha uma cópia em RAM, atualize a cópia com AND/OR e escreva o byte completo. O mesmo princípio vale para valores de display quando o software precisa recuperá-los.

```
; Exemplo conceitual: shadow de CPU_CFG em RAM
LDA CPU_CFG_SHADOW
AND A, #$8F       ; limpa CLK_B[2:0]
OR  A, #$40       ; código 100 = 1 MHz
OR  A, #$01       ; mantém RAM ativa
STA CPU_CFG_SHADOW
STA $F500
```

15.5 Polling de timers e periféricos

O WALL-8 não possui interrupções. BIOS e programas devem consultar SYSTEM_STATUS. Como detectar presença e disponibilidade são bits separados, a rotina deve distinguir periférico ausente de periférico presente sem evento.

- Presença ativa-baixo: TEST com a máscara e JZ significa conectado.
- KEY_READY ativo-alto: TEST 0x80 e JNZ significa dado pronto.
- Timer ativo-alto: leia o snapshot, processe todos os bits e só então escreva para limpar.

16. Exemplos de programação

16.1 Sequência segura de boot

```
#include "wall8.cpu"

; O arquivo da EPROM começa diretamente em 0x0000
RESET_ENTRY:
    JMP BIOS_RESIDENT

; Parte ligada/gravadada em 0x8000, com no máximo 8 KiB
BIOS_RESIDENT:
    STI $F400, #$00     ; desliga TONE_EN e NOISE_EN
    STI $F100, #$00     ; limpa display low
    STI $F101, #$00     ; limpa display high
    STI $F300, #$00     ; limpa os dois timers
    SP  $7FFF
    LDA #$00
    LDB #$00
    PTR $0000
    STI $F501, #$00     ; HEX + LEDs habilitados
    STI $F500, #$01     ; libera RAM, mantém bypass de 16 MHz

MAIN:
    JMP MAIN
```

> **Localização:** O trecho que executa depois de /BOOT_ROM_EN mudar para 1 deve estar realmente ligado e gravado em 0x8000-0x9FFF. Se o overlay nunca for liberado, toda a região ROM 0x0000-0x9FFF pode ser usada e não haverá RAM.

16.2 Leitura do teclado

```
WAIT_KEY:
    LDA $F300
    TEST A, #$40
    JNZ KEYBOARD_ABSENT

    TEST A, #$80
    JZ WAIT_KEY

    LDA $F000          ; leitura assertará /KEY_RD
    ; A contém o byte
    JMP KEY_RECEIVED
```

Se o teclado estiver presente, mas nenhuma tecla chegar até um timeout, a BIOS pode informar 'No Key'. Se /KEYBOARD_DETECT estiver em 1, deve informar 'No Present' e não aguardar o timeout.

16.3 Polling do timer

```
WAIT_500MS:
    LDA $F300
    TEST A, #$02
    JZ WAIT_500MS

    ; Salve/processe também o bit TIMER_1MS se necessário
    STI $F300, #$00
    ; passou um ou mais períodos de 500 ms
```

16.4 Display de quatro dígitos

```
; Exibe 0x1234 no modo HEX
STI $F501, #$00
STI $F100, #$34
STI $F101, #$12

; Carrega pares de códigos CHAR: A/b e C/d
; 1=A, 2=b, 3=C, 4=d
STI $F501, #$01
```

```
STI $F100, #$12
STI $F101, #$34
```

No modo HEX, cada nibble é um valor 0-F. No modo CHAR, cada nibble é um índice na tabela fixa de 16 símbolos gerada por 7Seg.py.

16.5 Geração de som

```
; Tom habilitado, seleção TONE=010
STI $F400, #$0A

; ... aguarda por software/timer ...

; Silêncio
STI $F400, #$00
```

SOUND_CFG substitui o byte completo. Para combinar ruído e tom, some os campos: bits 7:5 NOISE, bits 4:2 TONE, bit 1 TONE_EN e bit 0 NOISE_EN.

16.6 Percurso sequencial com PTR+

```
; Lê quatro bytes consecutivos a partir de 0x8000
PTR $8000
LDA [PTR+]          ; MEM[0x8000], PTR=0x8001
LDB [PTR+]          ; MEM[0x8001], PTR=0x8002
LDA [PTR+]          ; MEM[0x8002], PTR=0x8003
LDB [PTR+]          ; MEM[0x8003], PTR=0x8004
```

> **Um único PTR:** A ISA possui somente um ponteiro geral. Uma cópia entre duas regiões arbitrárias exige manter um dos endereços em RAM/código, usar stores absolutos ou adotar uma rotina especializada; não existe instrução de memória para memória.

17. BIOS, diagnóstico e fluxo de validação

17.1 Responsabilidades mínimas da BIOS

- Entrar diretamente pela EPROM em 0x0000 e, se a RAM for necessária, transferir a execução para a região residente em 0x8000-0x9FFF.
- Desligar o som e inicializar ambos os bytes de display.
- Limpar timers e inicializar SP, A, B e PTR quando usados.
- Testar RAM antes de depender dela para estruturas críticas.
- Detectar teclado, vídeo e cartucho pelos bits ativos-baixo.
- Liberar o overlay somente enquanto executa na região residente; alternativamente, mantê-lo ativo para operar com 40 KiB de ROM e sem RAM.
- Oferecer rotinas básicas de teclado, display, tempo, som e carga de cartucho.

17.2 Ordem de diagnóstico sugerida

Etapa	Bloco	Critério
1	`Alimentação`	5 V, polaridade, consumo, aquecimento e reset.
2	`Clock manual`	Ver cada T-state e confirmar T0/T1.
3	`PC/IR/micro-ROM`	Fetch repetível; opcode e linhas de controle corretos.
4	`Barramentos/MAR`	Uma única fonte por vez; endereço estável.
5	`RAM/BIOS`	Leitura, escrita e transição do overlay.
6	`A/B/ALU/flags`	Vetores de ADD/SUB/logic/shift e jumps.
7	`PTR/SP/CALL/RET`	Contagem, ordem da pilha e retorno.
8	`Timers/status`	Latch, leitura e reset conjunto.
9	`Display/som`	Inicialização segura e saídas.
10	`Módulos externos`	Detect, timing e tri-state.

17.3 Verificação do microcódigo

14. Compare cada .bin com 65.536 bytes e com o checksum esperado.
15. Em T0, confirme PC_OUT e MAR_LOAD; em T1, MEMORY_OE, IR_LOAD, MAR_OUT e PC_INC.
16. Verifique que todo opcode definido chega a STEP_RESET no ciclo documentado.
17. Procure conflitos de fontes no Data Bus e no Internal Address Bus.
18. Para cada jump, percorra as 16 combinações Z/C/N/V e confirme exatamente oito tomadas.
19. Teste CALL/RET com SP conhecido e confira bytes high/low na RAM.

20. Teste as formas imediatas com o outro registrador em valor sentinela para confirmar a restauração via TMP.

18. Estado do projeto, limitações e notas de implementação

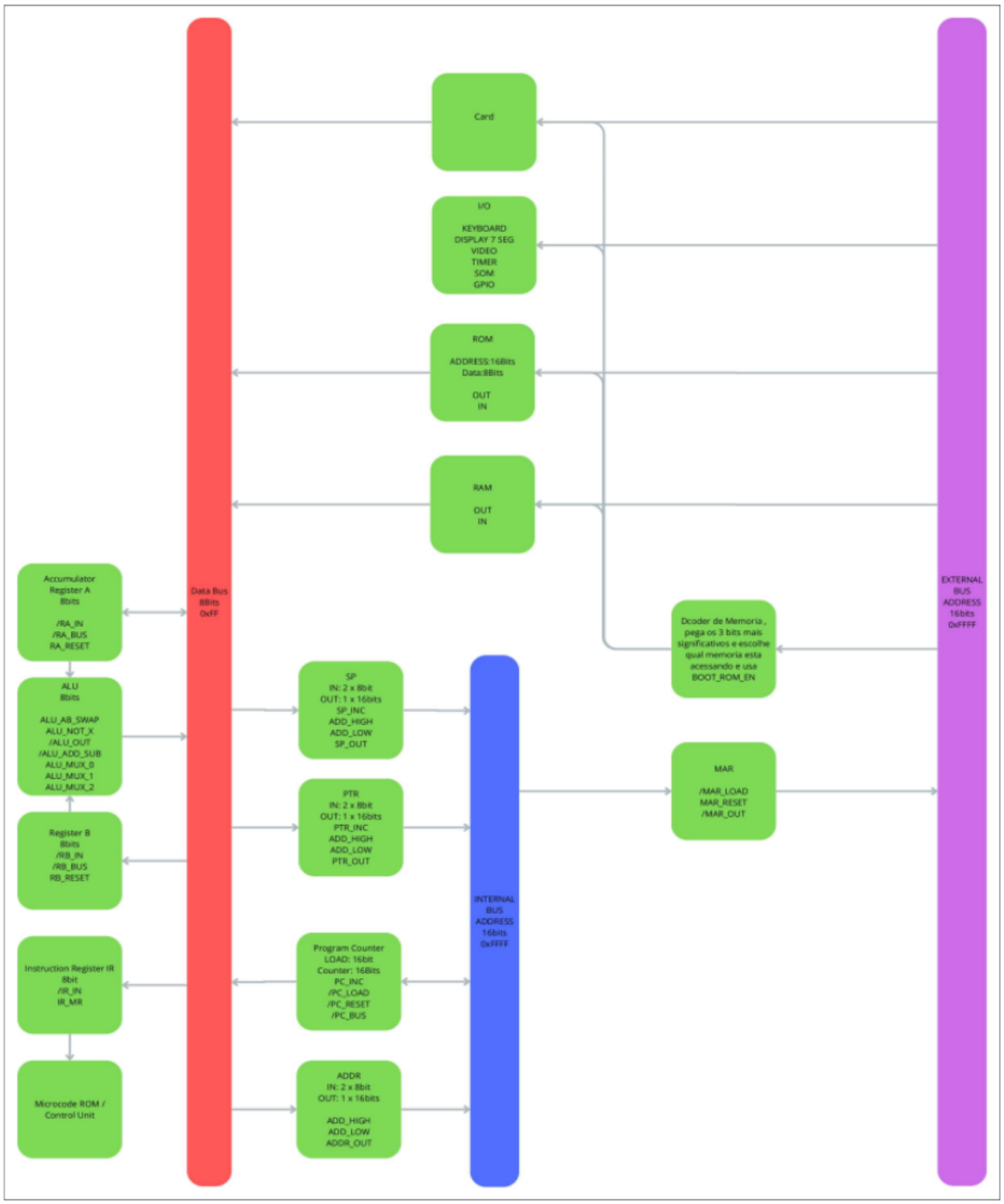
18.1 Recursos confirmados, reservados e futuros

Estado	Recursos
Confirmado	CPU microcodificada, 76 opcodes, RAM/ROM/CART/I/O, PTR, SP, stack, timers, teclado, display HEX/CHAR, som e contrato CPU-GPU.
Reservado	Y7 e detalhes internos da implementação da GPU além do contrato de registradores.
A validar	Operação física a 16 MHz, consumo, margens dos módulos externos, áudio, ESD, integridade de clock e hot-plug proibido.

18.2 Checklist antes da fabricação

- Confirmar footprint, orientação e pinagem de todos os 27C512, 62256, 74HC377, 74HC191/161 e conectores.
- Montar U117 como 74HC541 e manter a lista de materiais coerente com a família HC/HCT adotada.
- Confirmar que a imagem da BIOS ocupa os offsets corretos e que toda rotina executada após liberar o overlay cabe em 0x8000-0x9FFF.
- Gerar, dimensionar e conferir as cinco ROMs de microcódigo; programar na ordem correta.
- Gerar e validar display_low.bin e display_high.bin com 7Seg(1).py; ambos devem ter 65.536 bytes e conteúdo idêntico.
- Medir clock, reset, /ROM_MICRO_OE e todos os ramos após montagem parcial.
- Revisar contenção de barramento em cada ciclo de leitura/escrita e nos módulos externos.
- Calcular orçamento de corrente com LEDs, áudio, cartucho e GPU; dimensionar fonte e conectores.
- Confirmar que J3 bridge não possui ligação ao GND e que J6 deriva apenas de AUDIO_MIX.
- Executar teste de continuidade, shorts entre planos, alimentação limitada em corrente e bring-up por clock manual.

19. Apêndices – Anexos



Apêndice A - Microsequências por opcode

Todas as sequências começam com T0: PC_OUT + MAR_LOAD e T1: PC_INC + MAR_OUT + IR_LOAD + MEMORY_OE. A tabela mostra T2 em diante. Os nomes significam 'sinal assertado', mesmo quando a linha elétrica é ativa em nível baixo.

Opcode	Instrução	Ciclos	Microsteps após T1
0x00	NOP	3	T2: STEP_RESET
0x01	HLT	3	T2: HALT (T3 também está programado, mas normalmente não é alcançado)
0x10	LDA #$xx	5	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T4: STEP_RESET
0x11	LDB #$xx	5	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T4: STEP_RESET
0x12	LDA $addr	9	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: MAR_LOAD + ADDR_OUT T7: MAR_OUT + RA_LOAD + MEMORY_OE T8: STEP_RESET
0x13	LDB $addr	9	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: MAR_LOAD + ADDR_OUT T7: MAR_OUT + RB_LOAD + MEMORY_OE T8: STEP_RESET
0x14	STA $addr	10	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: MAR_LOAD + ADDR_OUT T7: MAR_OUT + RA_OUT T8: MAR_OUT + RA_OUT + MEMORY_WE T9: STEP_RESET
0x15	STB $addr	10	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: MAR_LOAD + ADDR_OUT T7: MAR_OUT + RB_OUT T8: MAR_OUT + RB_OUT + MEMORY_WE T9: STEP_RESET
0x16	MOV A, B	4	T2: RA_LOAD + RB_OUT T3: STEP_RESET
0x17	MOV B, A	4	T2: RA_OUT + RB_LOAD T3: STEP_RESET
0x18	LDA [PTR]	5	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RA_LOAD + MEMORY_OE T4: STEP_RESET
0x19	LDB [PTR]	5	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RB_LOAD + MEMORY_OE T4: STEP_RESET
0x1A	STA [PTR]	6	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RA_OUT T4: MAR_OUT + RA_OUT + MEMORY_WE T5: STEP_RESET
0x1B	STB [PTR]	6	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RB_OUT T4: MAR_OUT + RB_OUT + MEMORY_WE T5: STEP_RESET
0x20	PTR $addr	7	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + MEMORY_OE + PTR_L_LOAD T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + MEMORY_OE + PTR_H_LOAD T6: STEP_RESET

Opcode	Instrução	Ciclos	Microsteps após T1
0x21	INC PTR	4	T2: PTR_EN T3: STEP_RESET
0x22	DEC PTR	4	T2: PTR_EN + PTR_DOWN T3: STEP_RESET
0x23	LDA [PTR+]	6	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RA_LOAD + MEMORY_OE T4: PTR_EN T5: STEP_RESET
0x24	LDB [PTR+]	6	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RB_LOAD + MEMORY_OE T4: PTR_EN T5: STEP_RESET
0x25	STA [PTR+]	7	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RA_OUT T4: MAR_OUT + RA_OUT + MEMORY_WE T5: PTR_EN T6: STEP_RESET
0x26	STB [PTR+]	7	T2: MAR_LOAD + PTR_OUT T3: MAR_OUT + RB_OUT T4: MAR_OUT + RB_OUT + MEMORY_WE T5: PTR_EN T6: STEP_RESET
0x27	STI $addr, #$xx	12	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_OUT + MAR_LOAD T7: PC_INC + MAR_OUT + MEMORY_OE + TMP_LOAD T8: MAR_LOAD + ADDR_OUT T9: MAR_OUT + TMP_OUT T10: MAR_OUT + MEMORY_WE + TMP_OUT T11: STEP_RESET
0x30	ADD A, B	4	T2: RA_LOAD + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x31	ADD B, A	4	T2: RB_LOAD + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x32	SUB A, B	4	T2: RA_LOAD + ALU_SUB + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x33	SUB B, A	4	T2: RB_LOAD + ALU_AB_SWAP + ALU_SUB + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x34	AND A, B	4	T2: RA_LOAD + ALU_MUX_0 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x35	AND B, A	4	T2: RB_LOAD + ALU_MUX_0 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x36	OR A, B	4	T2: RA_LOAD + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x37	OR B, A	4	T2: RB_LOAD + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x38	XOR A, B	4	T2: RA_LOAD + ALU_MUX_0 + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x39	XOR B, A	4	T2: RB_LOAD + ALU_MUX_0 + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x3A	NOT A	4	T2: RA_LOAD + ALU_NOT_X + ALU_MUX_0 + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x3B	NOT B	4	T2: RB_LOAD + ALU_AB_SWAP + ALU_NOT_X + ALU_MUX_0 + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x3C	SHL A	4	T2: RA_LOAD + ALU_MUX_0 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x3D	SHR A	4	T2: RA_LOAD + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x3E	SHL B	4	T2: RB_LOAD + ALU_AB_SWAP + ALU_MUX_0 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x3F	SHR B	4	T2: RB_LOAD + ALU_AB_SWAP + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET

Opcode	Instrução	Ciclos	Microsteps após T1
0x40	CMP A, B	4	T2: ALU_SUB + FLAGS_LOAD T3: STEP_RESET
0x41	CMP B, A	4	T2: ALU_AB_SWAP + ALU_SUB + FLAGS_LOAD T3: STEP_RESET
0x42	INC A	4	T2: RA_LOAD + ALU_MUX_1 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x43	DEC A	4	T2: RA_LOAD + ALU_SUB + ALU_MUX_1 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x44	INC B	4	T2: RB_LOAD + ALU_AB_SWAP + ALU_MUX_1 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x45	DEC B	4	T2: RB_LOAD + ALU_AB_SWAP + ALU_SUB + ALU_MUX_1 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x46	CLR A	4	T2: RA_LOAD + ALU_MUX_0 + ALU_MUX_1 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x47	CLR B	4	T2: RB_LOAD + ALU_MUX_0 + ALU_MUX_1 + ALU_MUX_2 + ALU_OUT + FLAGS_LOAD T3: STEP_RESET
0x50	JMP $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT T7: STEP_RESET
0x51	JZ $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se Z=1] T7: STEP_RESET
0x52	JNZ $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se Z=0] T7: STEP_RESET
0x53	JC $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se C=1] T7: STEP_RESET
0x54	JNC $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se C=0] T7: STEP_RESET
0x55	JN $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se N=1] T7: STEP_RESET
0x56	JP $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se N=0] T7: STEP_RESET
0x57	JV $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se V=1] T7: STEP_RESET
0x58	JNV $addr	8	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: PC_LOAD + ADDR_OUT [se V=0] T7: STEP_RESET

Opcode	Instrução	Ciclos	Microsteps após T1
0x59	CALL $addr	16	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + ADDR_H_LOAD + MEMORY_OE T6: SP_EN + SP_DOWN T7: MAR_LOAD + SP_OUT T8: MAR_OUT + PC_OUT_H_DATA T9: MAR_OUT + MEMORY_WE + PC_OUT_H_DATA T10: SP_EN + SP_DOWN T11: MAR_LOAD + SP_OUT T12: MAR_OUT + PC_OUT_L_DATA T13: MAR_OUT + MEMORY_WE + PC_OUT_L_DATA T14: PC_LOAD + ADDR_OUT T15: STEP_RESET
0x5A	RET	10	T2: MAR_LOAD + SP_OUT T3: MAR_OUT + ADDR_L_LOAD + MEMORY_OE T4: SP_EN T5: MAR_LOAD + SP_OUT T6: MAR_OUT + ADDR_H_LOAD + MEMORY_OE T7: SP_EN T8: PC_LOAD + ADDR_OUT T9: STEP_RESET
0x5B	PUSH A	7	T2: SP_EN + SP_DOWN T3: MAR_LOAD + SP_OUT T4: MAR_OUT + RA_OUT T5: MAR_OUT + RA_OUT + MEMORY_WE T6: STEP_RESET
0x5C	PUSH B	7	T2: SP_EN + SP_DOWN T3: MAR_LOAD + SP_OUT T4: MAR_OUT + RB_OUT T5: MAR_OUT + RB_OUT + MEMORY_WE T6: STEP_RESET
0x5D	POP A	6	T2: MAR_LOAD + SP_OUT T3: MAR_OUT + RA_LOAD + MEMORY_OE T4: SP_EN T5: STEP_RESET
0x5E	POP B	6	T2: MAR_LOAD + SP_OUT T3: MAR_OUT + RB_LOAD + MEMORY_OE T4: SP_EN T5: STEP_RESET
0x5F	SP $addr	7	T2: PC_OUT + MAR_LOAD T3: PC_INC + MAR_OUT + MEMORY_OE + SP_L_LOAD T4: PC_OUT + MAR_LOAD T5: PC_INC + MAR_OUT + MEMORY_OE + SP_H_LOAD T6: STEP_RESET
0x60	ADD A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: RA_LOAD + ALU_OUT + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET
0x61	ADD B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: RB_LOAD + ALU_OUT + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET
0x62	SUB A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: RA_LOAD + ALU_SUB + ALU_OUT + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET
0x63	SUB B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: RB_LOAD + ALU_AB_SWAP + ALU_SUB + ALU_OUT + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET
0x64	AND A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: RA_LOAD + ALU_MUX_0 + ALU_OUT + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET

Opcode	Instrução	Ciclos	Microsteps após T1
0x65	AND B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: RB_LOAD + ALU_MUX_0 + ALU_OUT + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET
0x66	OR A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: RA_LOAD + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET
0x67	OR B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: RB_LOAD + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET
0x68	XOR A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: RA_LOAD + ALU_MUX_0 + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET
0x69	XOR B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: RB_LOAD + ALU_MUX_0 + ALU_MUX_1 + ALU_OUT + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET
0x6A	CMP A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: ALU_SUB + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET
0x6B	CMP B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: ALU_AB_SWAP + ALU_SUB + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET
0x6C	TEST A, #$xx	8	T2: RB_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RB_LOAD + MEMORY_OE T5: ALU_MUX_0 + FLAGS_LOAD T6: RB_LOAD + TMP_OUT T7: STEP_RESET
0x6D	TEST B, #$xx	8	T2: RA_OUT + TMP_LOAD T3: PC_OUT + MAR_LOAD T4: PC_INC + MAR_OUT + RA_LOAD + MEMORY_OE T5: ALU_MUX_0 + FLAGS_LOAD T6: RA_LOAD + TMP_OUT T7: STEP_RESET

Apêndice B - Glossário dos sinais de controle

ROM	Bit	Sinal	Ativo	Função
1	D0	PC_INC	H	Incrementa o Program Counter.
1	D1	/PC_LOAD	L	Carrega PC a partir do Internal Address Bus.
1	D2	/PC_OUT	L	PC dirige o Internal Address Bus.
1	D3	/MAR_LOAD	L	MAR captura o Internal Address Bus.
1	D4	/MAR_OUT	L	MAR dirige o External Address Bus.
1	D5	/IR_LOAD	L	IR captura o Data Bus.
1	D6	/RA_LOAD	L	A captura o Data Bus/saída da ALU.
1	D7	/RA_OUT	L	A dirige o Data Bus.
2	D0	/RB_LOAD	L	B captura o Data Bus/saída da ALU.
2	D1	/RB_OUT	L	B dirige o Data Bus.
2	D2	/ADDR_L_LOAD	L	Carrega o byte baixo de ADDR.
2	D3	/ADDR_H_LOAD	L	Carrega o byte alto de ADDR.
2	D4	/ADDR_OUT	L	ADDR dirige o Internal Address Bus.
2	D5	ALU_AB_SWAP	H	Troca A/B na seleção X/Y da ALU.
2	D6	ALU_NOT_X	H	Seleciona NOT X no caminho XOR/NOT.
2	D7	ALU_SUB	H	Seleciona SUB ou DEC nos blocos aritméticos.
3	D0	ALU_MUX_0	H	Bit 0 da seleção de função da ALU.
3	D1	ALU_MUX_1	H	Bit 1 da seleção de função da ALU.
3	D2	ALU_MUX_2	H	Bit 2 da seleção de função da ALU.
3	D3	/ALU_OUT	L	Saída da ALU dirige o Data Bus.
3	D4	/MEMORY_OE	L	Habilita leitura de memória/I/O.
3	D5	/MEMORY_WE	L	Habilita escrita em memória/I/O.
3	D6	/FLAGS_LOAD	L	Captura Z, C, N e V.
3	D7	/STEP_RESET	L	Carrega zero no Step Counter.
4	D0	/HALT	L	Bloqueia o clock da CPU.
4	D1	/PTR_EN	L	Habilita contagem de PTR.
4	D2	/PTR_L_LOAD	L	Carrega byte baixo de PTR.
4	D3	/PTR_H_LOAD	L	Carrega byte alto de PTR.
4	D4	/PTR_OUT	L	PTR dirige o Internal Address Bus.
4	D5	PTR_DOWN	H	Seleciona decremento; baixo seleciona incremento.
4	D6	/TMP_LOAD	L	TMP captura o Data Bus.
4	D7	/TMP_OUT	L	TMP dirige o Data Bus.
5	D0	/SP_EN	L	Habilita contagem de SP.
5	D1	/SP_L_LOAD	L	Carrega byte baixo de SP.
5	D2	/SP_H_LOAD	L	Carrega byte alto de SP.
5	D3	/SP_OUT	L	SP dirige o Internal Address Bus.

ROM	Bit	Sinal	Ativo	Função
5	D4	SP_DOWN	H	Seleciona decremento; baixo seleciona incremento.
5	D5	/PC_OUT_L_DATA	L	Byte baixo do PC dirige o Data Bus.
5	D6	/PC_OUT_H_DATA	L	Byte alto do PC dirige o Data Bus.
5	D7	NC	–	Saída reservada/não conectada.

> **ROM5 D7:** O gerador denomina esse bit NA e o esquema o mostra sem função. O idle 0x6F deixa D7 em 0, mas a saída não deve ser conectada ou usada como controle.

Apêndice C - Baseline e checksums

Checksums SHA-256 dos arquivos-fonte usados para gerar esta revisão:

Arquivo	Bytes	SHA-256
Hardware_Completo.pdf	494.295	35a2c299b1c3cd744e196c7d4014cc5e346b4b2a3ea7224ce094d8ccf821a12f
wall8_apresentacao_arquitetura(20260808-152303).md	23.847	6963e70c17a05e3307dd04e0fb49c07120061b39b2f5e53d08ce9b00acb63d3b
wall8_instrucoes(20260808-152303).md	11.611	ea55f5f671cf1e35d1cc22ff959c8fa1e1f7b0f07ec071467f22aab4b48daadc
wall8.cpu	8.008	8a2ddf99074b2e8bda75fab7e88aa6c9fb3d60cae9287ce18c9cadcf1284dbfa
Gerador_Microcodigo.py	34.048	326f5054a3d7c0de81fea9e870233718d4327c2ed10b09db008fc4b397e8aeb0
7Seg.py	3.610	00389db045240fe840d10c2cb8be80f1209ad24825ae66404fcab0955b1d4645
wall8_placa_video.md	66.479	6093d455f36ae9a6adc79d916fe310fc6ab2ce6b1aa45b031d83befc312b351a

Imagens produzidas pela baseline Gerador_Microcodigo(4).py:

Imagem	Bytes	SHA-256
microcode_rom1.bin	65536	e74c85458c90b83399596d507ddc01f098ee0286aedae5a39c54f09018f7856e
microcode_rom2.bin	65536	f804a589f22435537fc82bb0e860fd1c6dfc355c5e0f738db69434841af44202
microcode_rom3.bin	65536	539fea8e0a442bdd863285300f1a4d52e63d8002d0a9bfd479c60a4f7a8c4721
microcode_rom4.bin	65536	3689644e9ffc423f724bf96fba43ffc84371cb900e28410100d0d488575554c5
microcode_rom5.bin	65536	634f5c0ff6ab009d145c9c29000da2c6daf0ad9dd8ae0918a79c775f9b19afab

Apêndice D - Histórico de revisões

Revisão	Data	Descrição
1.0	08/08/2026	Primeira consolidação integral de hardware, microcódigo, ISA, I/O, pinagem, programação, limitações e área reservada para o esquema completo.
1.1	08/08/2026	Consolida clock, overlay da BIOS, som, display e U117; incorpora o contrato CPU-visível da GPU e o crédito ao customasm.

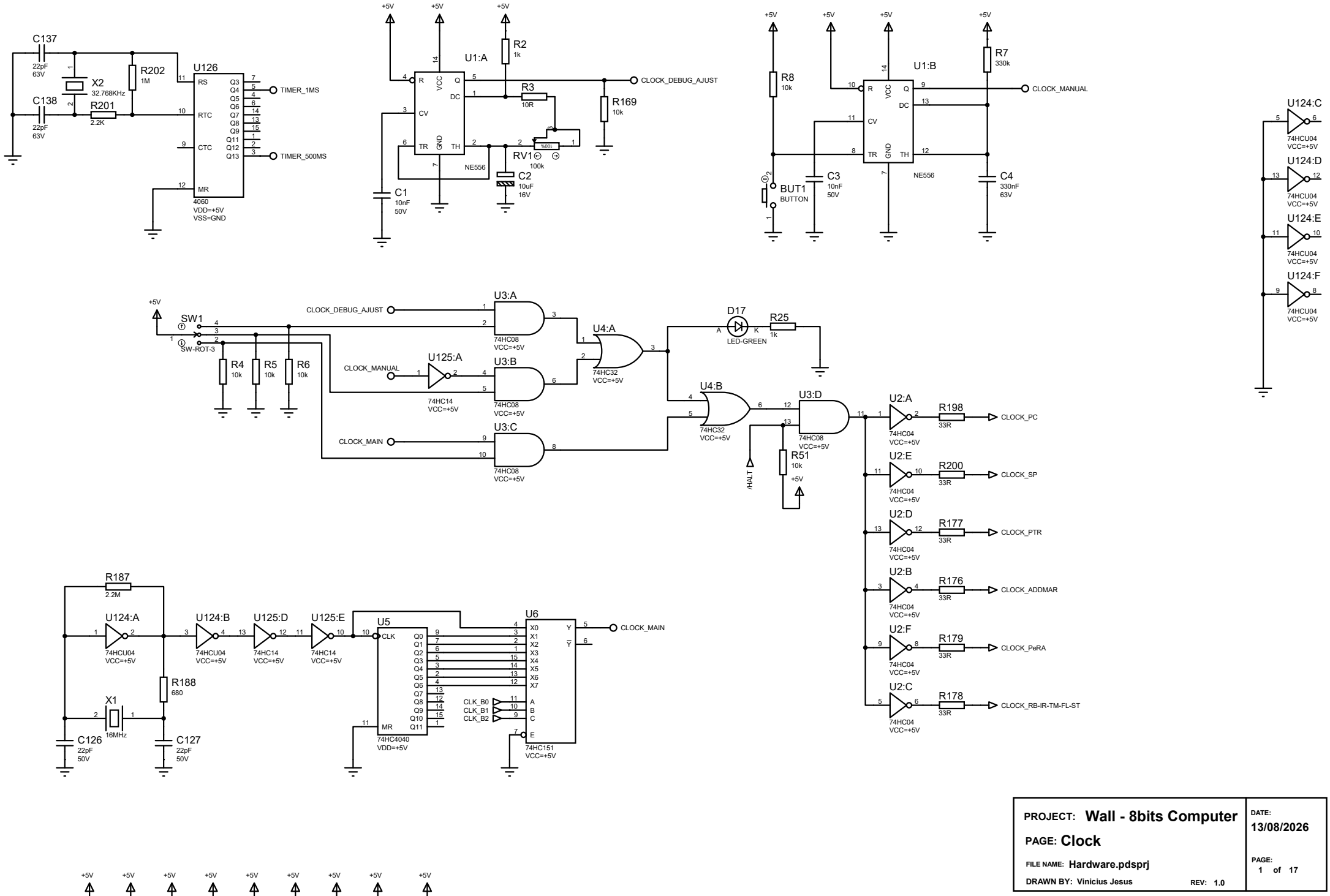
Fim do Manual de Referência Técnica WALL-8 - Revisão 1.1.

Anexo A — Esquema elétrico completo

Folha	Título esperado
1	Clock
2	PC - Program Counter
3	Register A / ACC
4	Register B
5	IR - Instruction Register
6	ALU
7	MAR - Memory Address Register
8	RAM Memory
9	Decode Memory / I/O / Sound / Status
10	ROM/BIOS Memory
11	Card Memory / J1
12	Display HEX
13	ADDR - Address Register
14	Unit Control / alimentação
15	PTR - Pointer Register
16	TMP - Temporary Register
17	SP - Stack Pointer

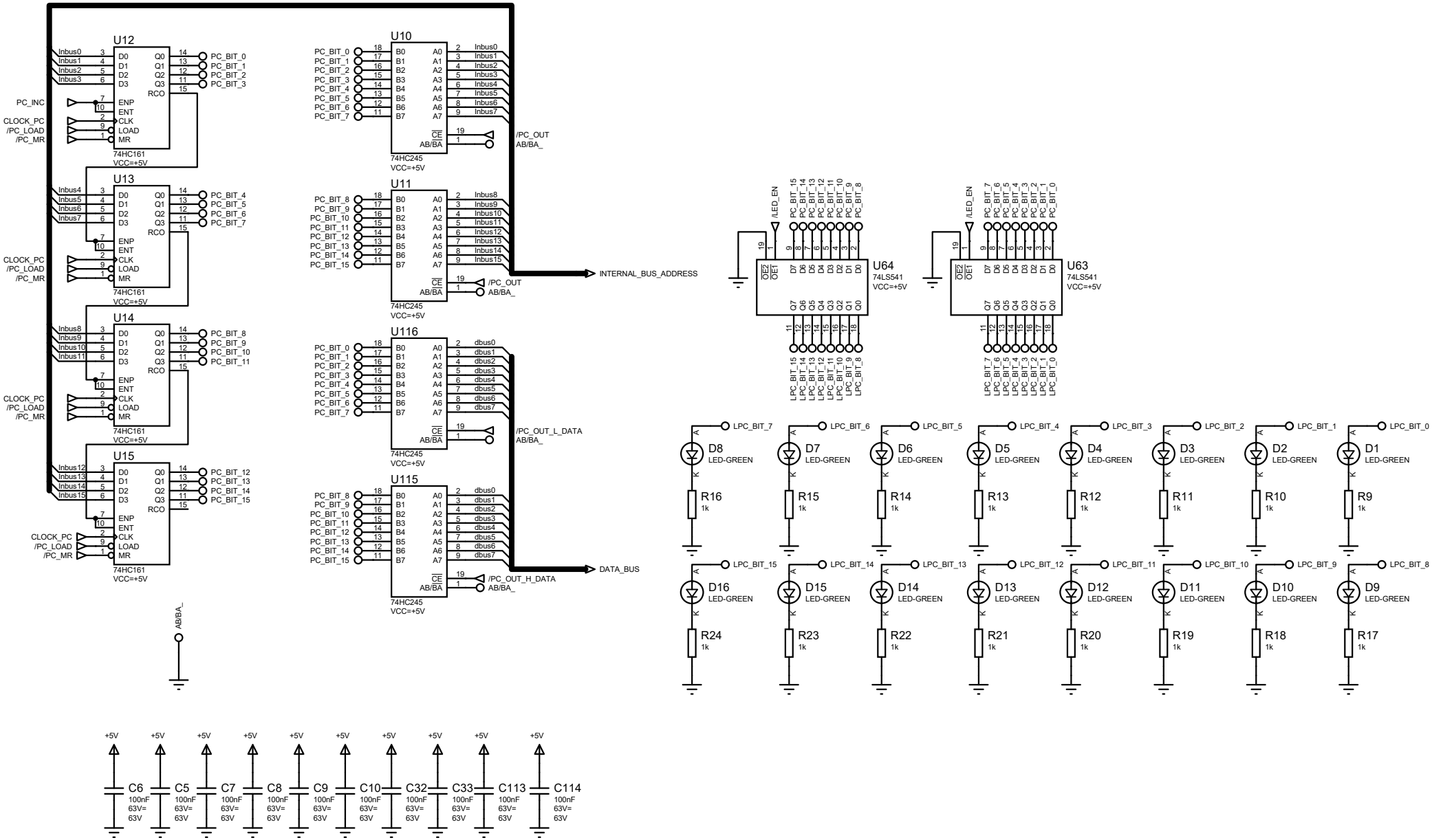
INÍCIO DO ANEXO DE ESQUEMAS

CLOCK



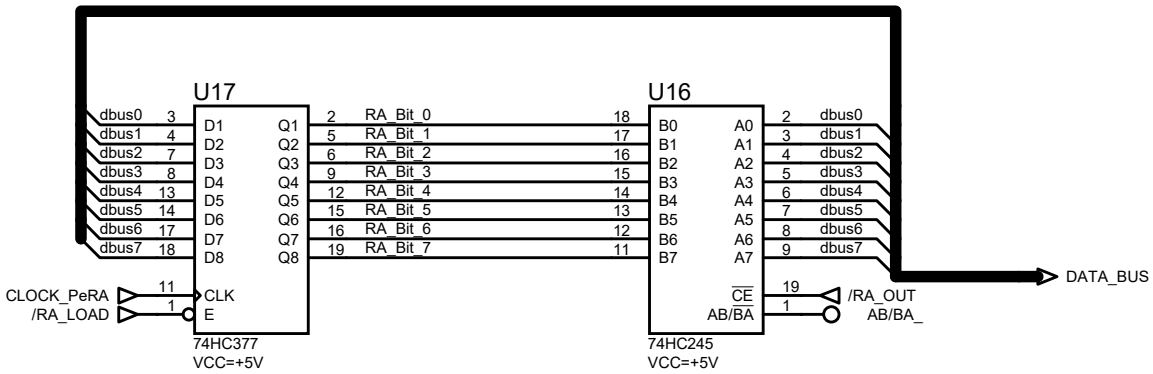
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **Clock**	PAGE: **1 of 17**
FILE NAME: **Hardware.pdsprj**	
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

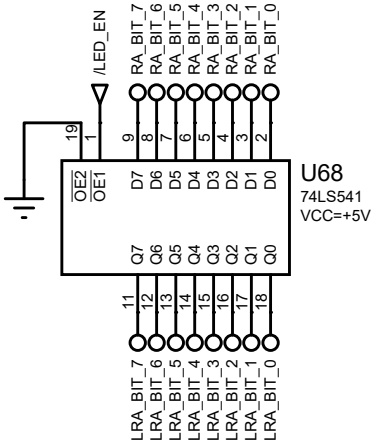
PC - Program Counter

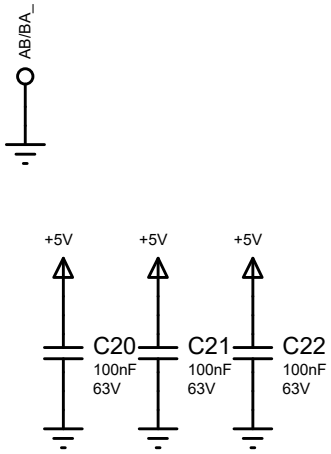


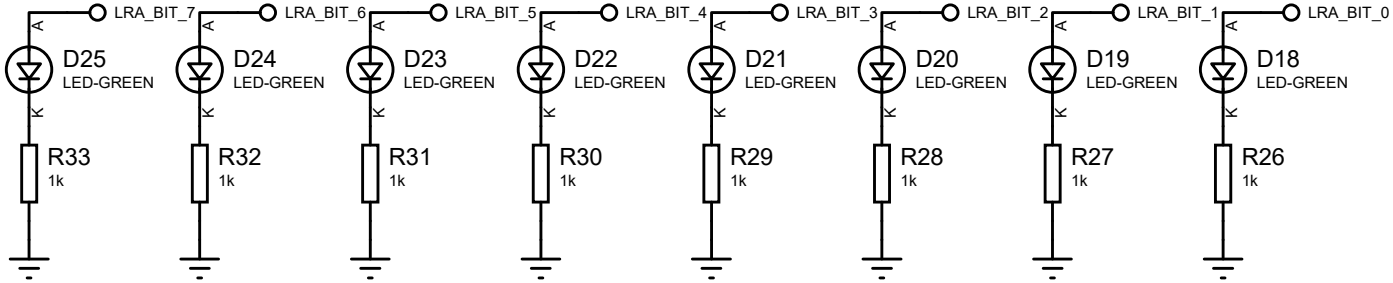
PROJECT: **Wall - 8bits Computer**		DATE: **13/08/2026**
PAGE: **Program Counter 16bits**		
FILE NAME: **Hardware.pdsprj**		PAGE: **2 of 17**
DRAWN BY: **Vinicius Jesus**	REV: **1.0**	

Register A



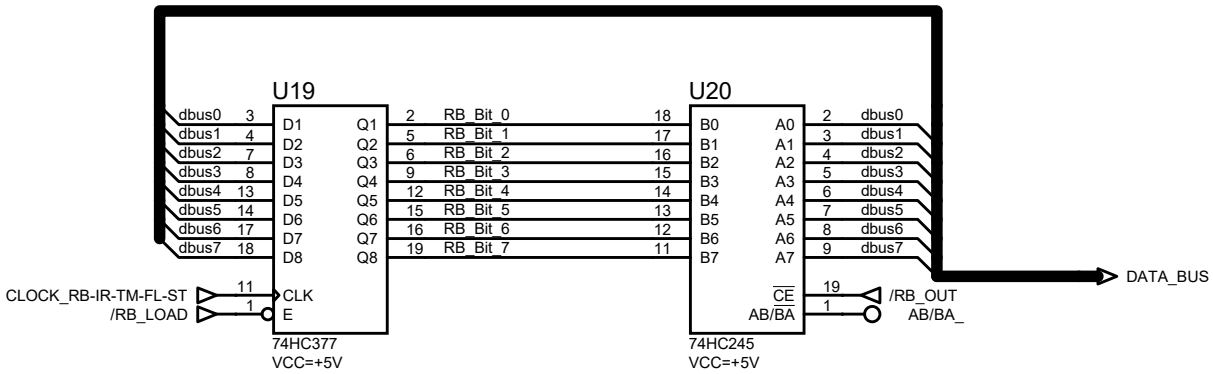


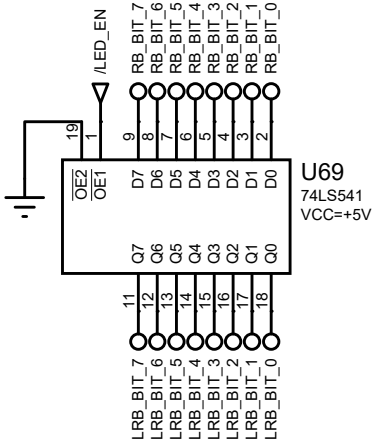


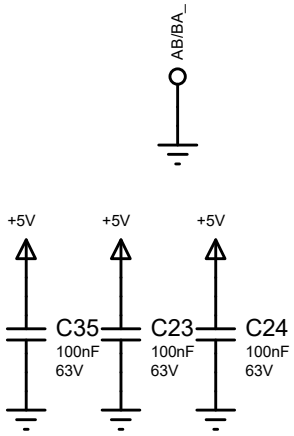


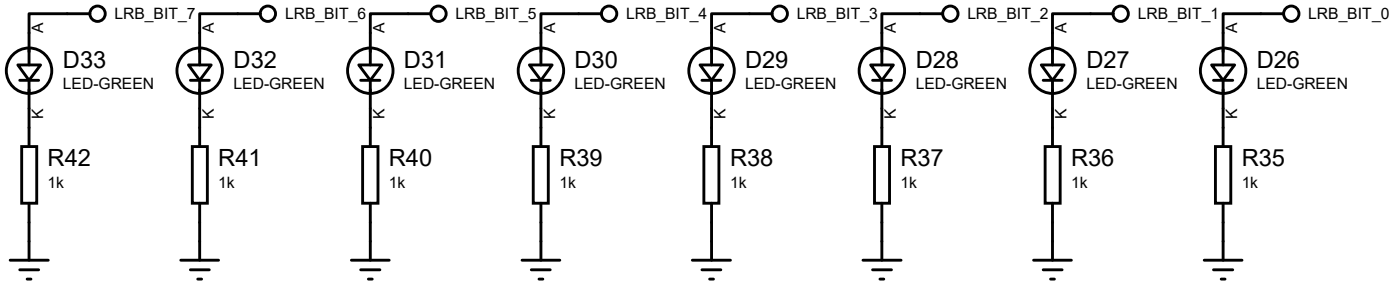
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **Registrador A , ACC**	
FILE NAME: **Hardware.pdsprj**	PAGE: **3 of 17**
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

Register B



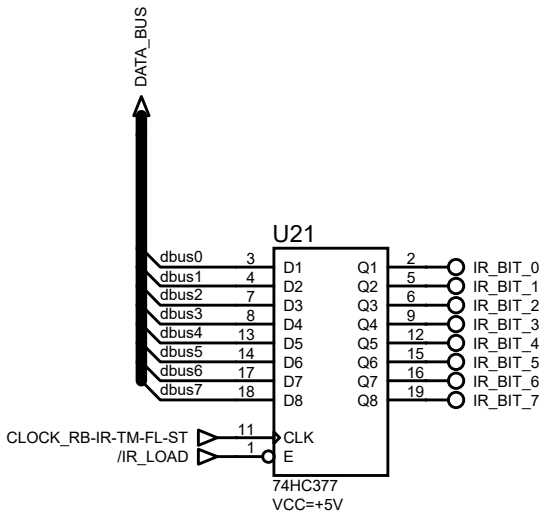


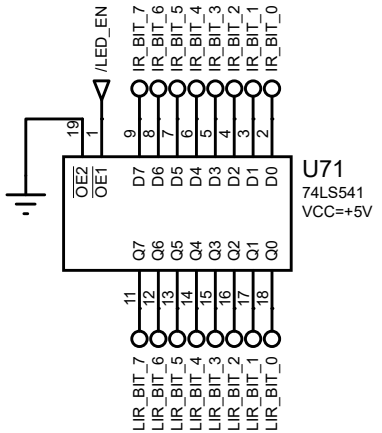


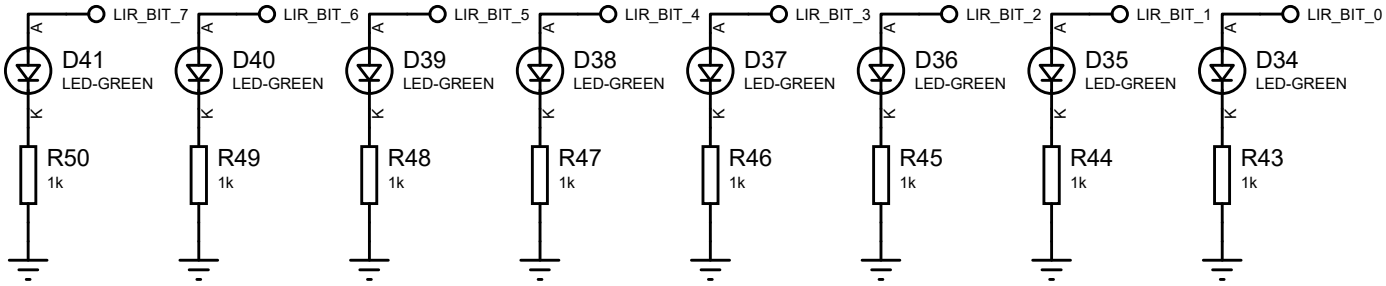


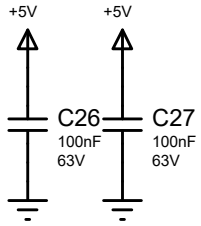
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **Registrador B**	
FILE NAME: **Hardware.pdsprj**	PAGE: **4 of 17**
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

IR - Instruction Register





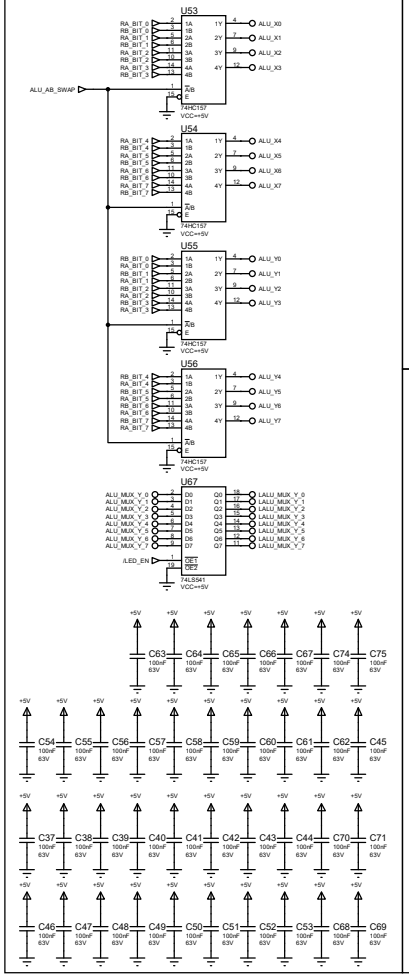




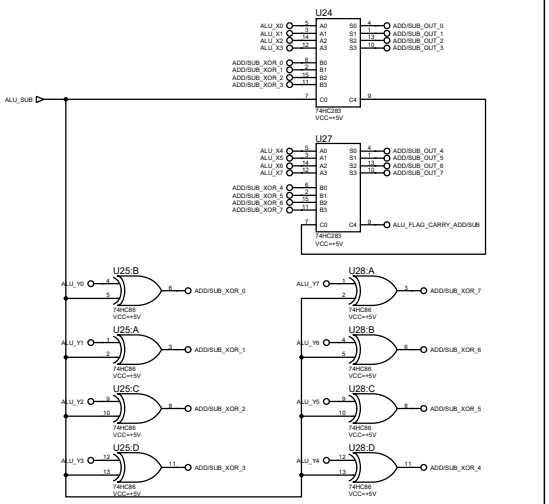
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **IR Instruction Register**	
FILE NAME: **Hardware.pdsprj**	PAGE: **5 of 17**
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

ALU
Arithmetic Logic Unit

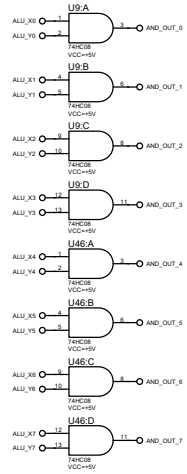
MUX



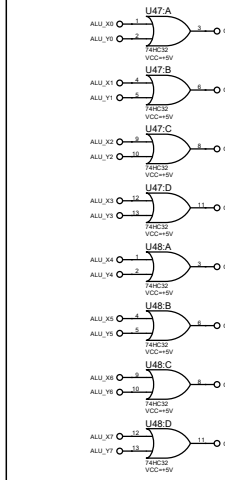
ADD/SUB X +/- Y 8Bits



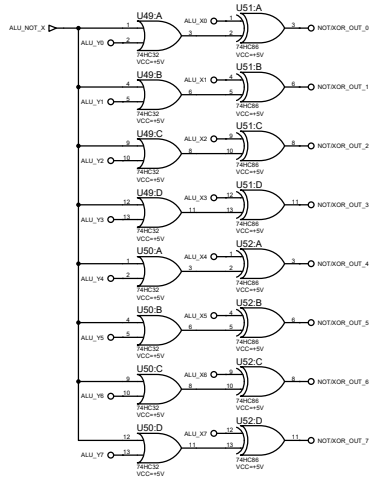
AND X - Y 8Bits



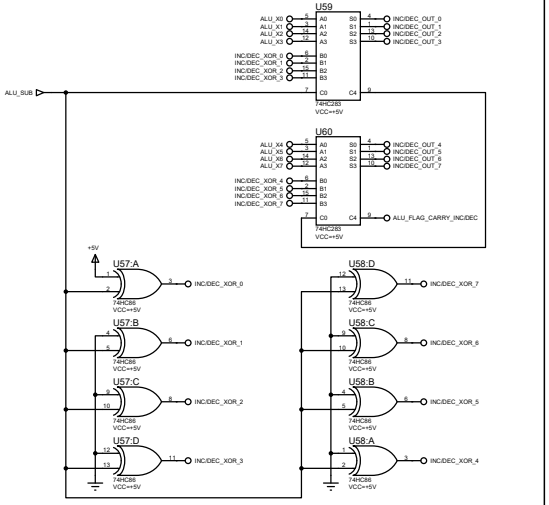
OR X - Y 8Bits



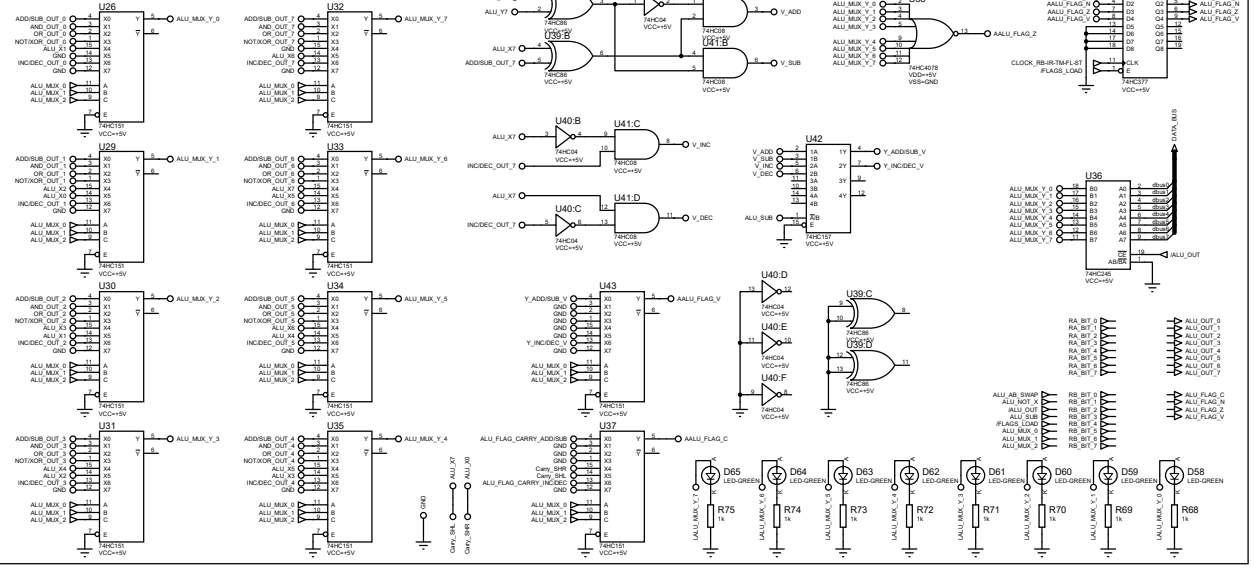
NOT/XOR X XOR Y NOT X 8Bits



INC/DEC X 8Bits



ADD/SUB X + Y 8Bits



---------- ALU WALL-8 ----------

ENTRADAS DE DADOS:
RA_BIT_0 ... RA_BIT_7 = Registrador A
RB_BIT_0 ... RB_BIT_7 = Registrador B

BIT 0 = menos significativo
BIT 7 = mais significativo

SELETOR DE ENTRADA A/B:

ALU_AB_SWAP = 0:
ALU_X = RA
ALU_Y = RB

ALU_AB_SWAP = 1:
ALU_X = RB
ALU_Y = RA

ENTRADAS DE CONTROLE:

ALU_MUX_0
ALU_MUX_1
ALU_MUX_2

ALU_NOT_X:
0 = XOR -> ALU_X XOR ALU_Y
1 = NOT -> NOT ALU_X

/ALU_ADD_SUB:
0 = ADD / INC
1 = SUB / DEC

/ALU_BUS:
0 = ALU_OUT habilitada no barramento
1 = ALU_OUT isolada do barramento

TABELA DO ALU_MUX:

ALU_MUX_2 ALU_MUX_1 ALU_MUX_0

000 = ADD/SUB
/ALU_ADD_SUB = 0 -> ALU_X + ALU_Y
/ALU_ADD_SUB = 1 -> ALU_X - ALU_Y

001 = AND
ALU_X AND ALU_Y

010 = OR
ALU_X OR ALU_Y

011 = XOR/NOT
ALU_NOT_X = 0 -> ALU_X XOR ALU_Y
ALU_NOT_X = 1 -> NOT ALU_X

100 = SHR
ALU_OUT = ALU_X >> 1
Carry = ALU_X0

101 = SHL
ALU_OUT = ALU_X << 1
Carry = ALU_X7

110 = INC/DEC
/ALU_ADD_SUB = 0 -> ALU_X + 1
/ALU_ADD_SUB = 1 -> ALU_X - 1

111 = ZERO
ALU_OUT = 00000000

FLAGS:

ALU_FLAG_C:
Carry / Not Borrow / bit deslocado

ALU_FLAG_Z:
1 se ALU_OUT = 00000000
0 se ALU_OUT tiver qualquer bit em 1

ALU_FLAG_N:
ALU_FLAG_N = ALU_OUT_7

ALU_FLAG_V:
Overflow em operação com sinal
Usada em ADD/SUB e INC/DEC
Nas demais operações = 0

CARRY POR OPERAÇÃO:

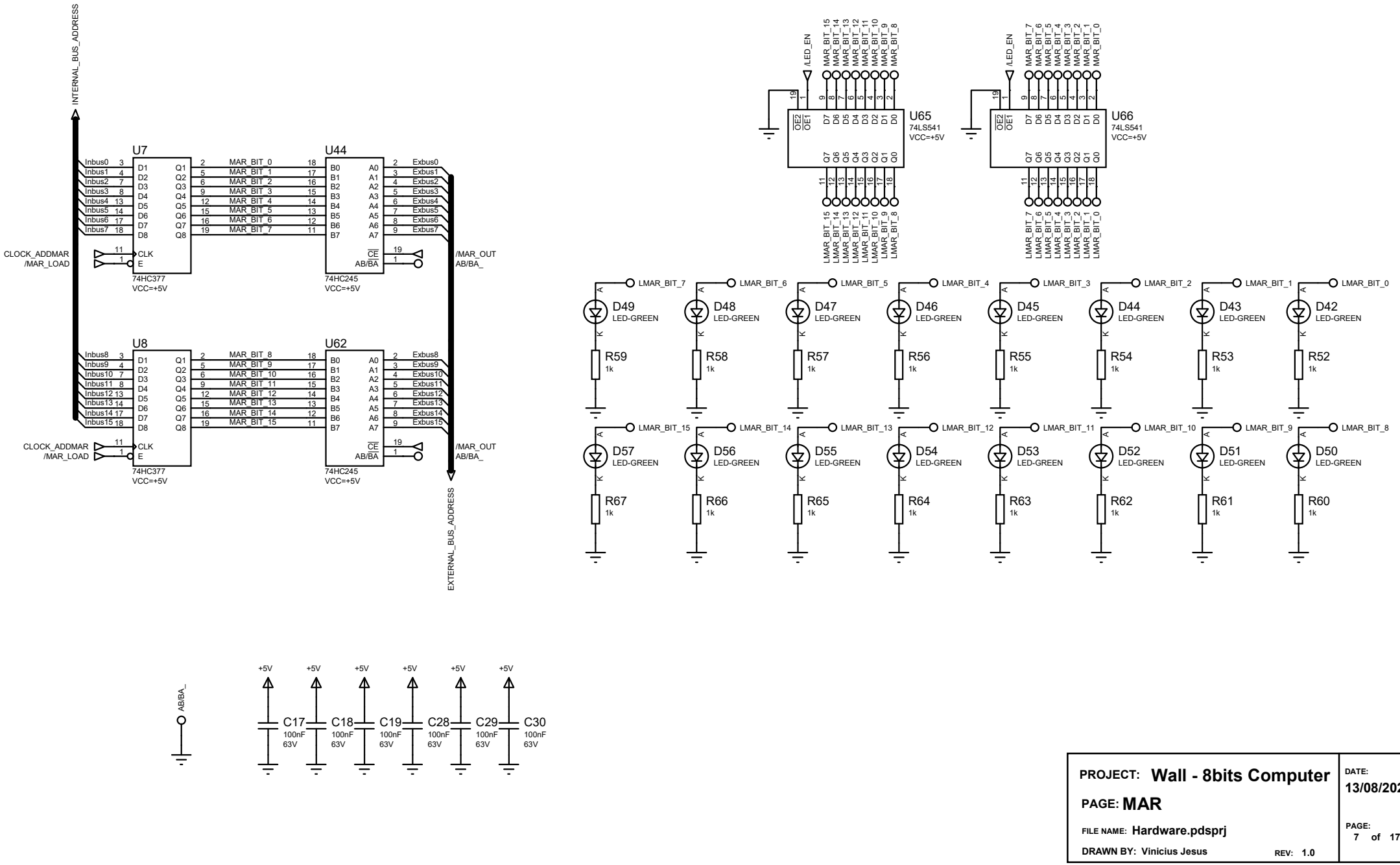
000 ADD/SUB = Carry do ADD/SUB
001 AND = 0
010 OR = 0
011 XOR/NOT = 0
100 SHR = ALU_X0
101 SHL = ALU_X7
110 INC/DEC = Carry do INC/DEC
111 ZERO = 0

OVERFLOW V POR OPERAÇÃO:

000 ADD/SUB = V do ADD/SUB
001 AND = 0
010 OR = 0
011 XOR/NOT = 0
100 SHR = 0
101 SHL = 0
110 INC/DEC = V do INC/DEC
111 ZERO = 0

PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **ALU**	PAGE: **6 of 17**
FILE NAME: **Hardware.pdsprj**	
DRAWN BY: **Vinicius Jesus**	REV: **1.0**

MAR - Memory Address Register

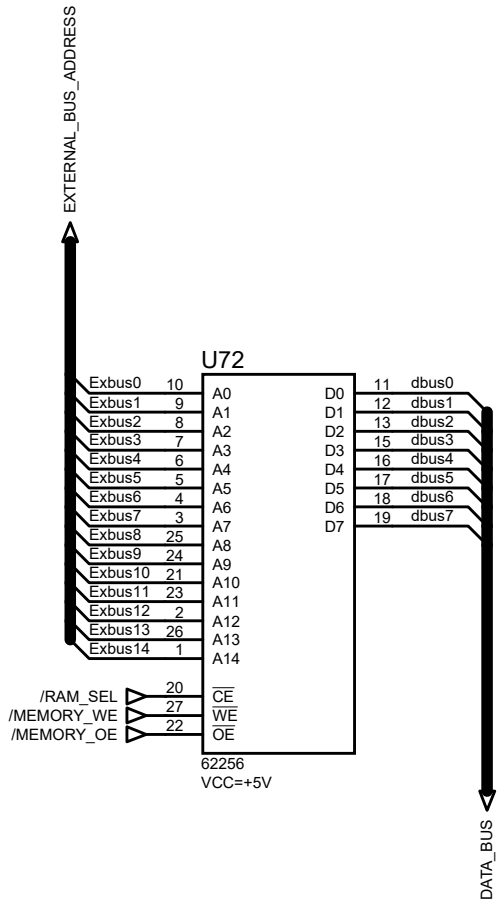


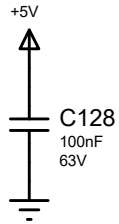
PROJECT: **Wall - 8bits Computer**		DATE: **13/08/2026**
PAGE: **MAR**		
FILE NAME: **Hardware.pdsprj**		PAGE: **7 of 17**
DRAWN BY: **Vinicius Jesus**	REV: **1.0**	

WALL-8 RAM MEMORY

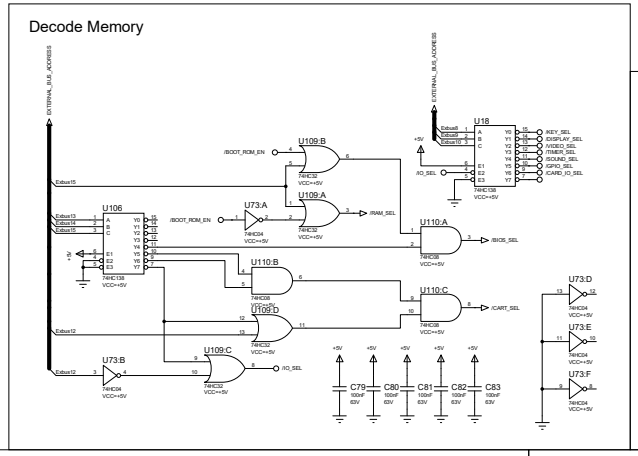
ADDRESS BUS: 16 bits
DATA BUS: 8 bits

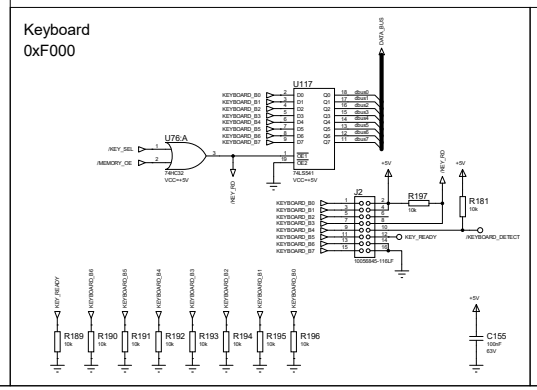
0x0000 - 0x7FFF RAM 32 KB 32768 bytes

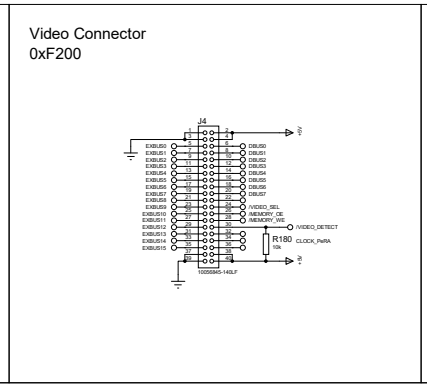


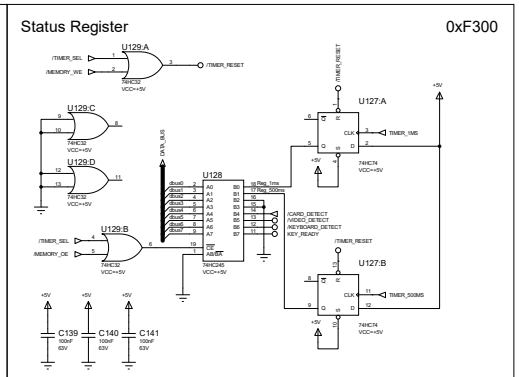


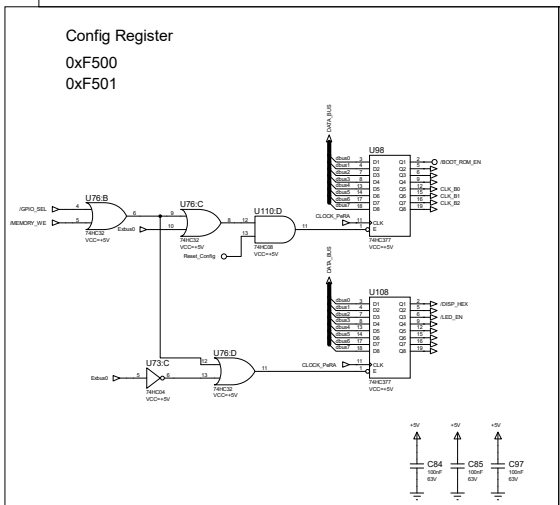
PROJECT: **Wall - 8bits Computer**		DATE: **13/08/2026**
PAGE: **RAM Memory**		
FILE NAME: **Hardware.pdsprj**		PAGE: **8 of 17**
DRAWN BY: **Vinicius Jesus**	REV: **1.0**	

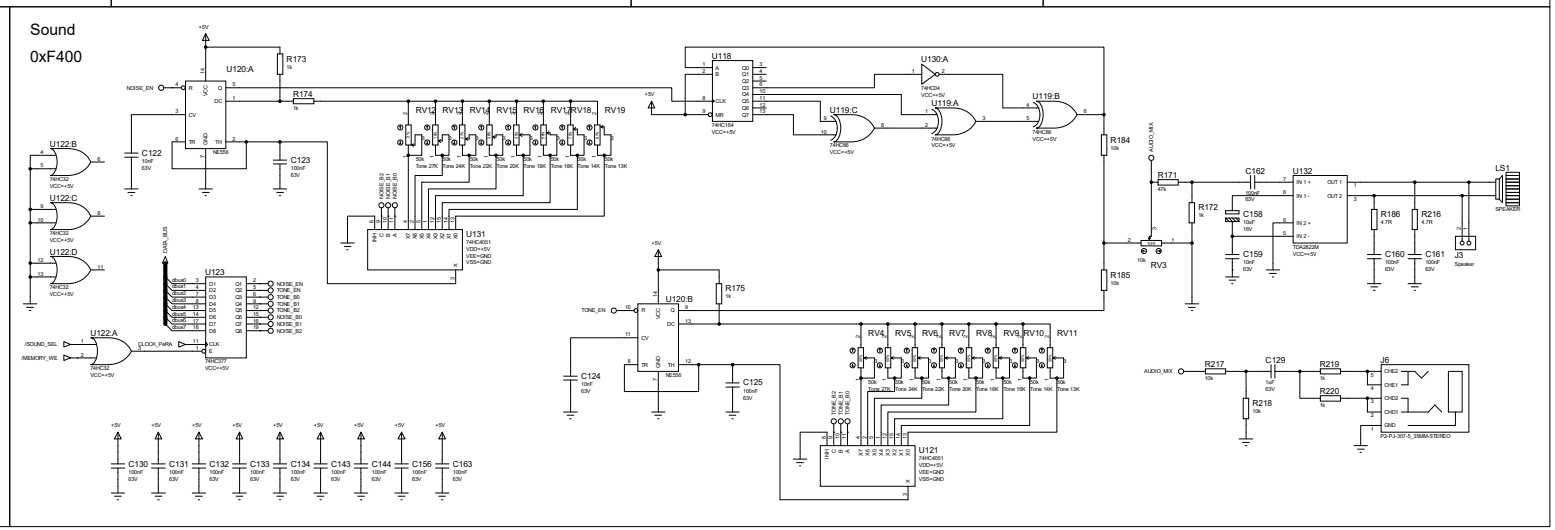












WALL-8 I/O MAP - 74HC138 SUB-DECODE

I/O = 0xF000-0xFFFF
BIN = 1111 xCBA xxxx xxxx

74HC138:
A = A8
B = A9
C = A10

CBA / A10 A9 A8:

000 -> Y0 /KEY_SEL
HEX: 0xF000-0xF0FF and 0xF800-0xF8FF
BIN: 1111 x000 xxxx xxxx

001 -> Y1 /DISPLAY_SEL
HEX: 0xF100-0xF1FF and 0xF900-0xF9FF
BIN: 1111 x001 xxxx xxxx

0xF100 = DISPLAY_LOW
0xF101 = DISPLAY_HIGH

010 -> Y2 /VIDEO_SEL
HEX: 0xF200-0xF2FF and 0xFA00-0xFAFF
BIN: 1111 x010 xxxx xxxx

011 -> Y3 /TIMER_SEL
HEX: 0xF300-0xF3FF and 0xFB00-0xFBFF
BIN: 1111 x011 xxxx xxxx

100 -> Y4 /SOUND_SEL
HEX: 0xF400-0xF4FF and 0xFC00-0xFCFF
BIN: 1111 x100 xxxx xxxx

SOUND_SEL

0xF400 = Bits de config Sound

BIT 0 = /NOISE_EN
BIT 1 = /TONE_EN
BIT 2 = NC
BIT 3 = NC
BIT 4 = TONE_B0
BIT 5 = TONE_B1
BIT 6 = TONE_B2
BIT 7 = NC

101 -> Y5 /GPIO_SEL
HEX: 0xF500-0xF5FF and 0xFD00-0xFDFF
BIN: 1111 x101 xxxx xxxx

0xF500 = Bits de config CPU

BIT 0 = /BOOT_ROM_EN
BIT 1 = NC
BIT 2 = NC
BIT 3 = NC
BIT 4 = /CLK_B0
BIT 5 = /CLK_B1
BIT 6 = /CLK_B2
BIT 7 = NC

0xF501 = Bits de config Perifericos

BIT 0 = /DISP_HEX
BIT 1 = NC
BIT 2 = /LED_EN
BIT 3 = NC
BIT 4 = NC
BIT 5 = NC
BIT 6 = NC
BIT 7 = NC

110 -> Y6 Não Usado
HEX: 0xF600-0xF6FF and 0xFE00-0xFEFF
BIN: 1111 x110 xxxx xxxx

111 -> Y7 Não Usado
HEX: 0xF700-0xF7FF and 0xFF00-0xFFFF
BIN: 1111 x111 xxxx xxxx

WALL-8 MEMORY MAP - 64 KB ADDRESS SPACE

ADDRESS BUS: 16 bits
DATA BUS: 8 bits

0x0000 - 0x7FFF RAM 32 KB 32768 bytes
0x8000 - 0x9FFF BIOS/ROM 8 KB 8192 bytes
0xA000 - 0xEFFF CART 20 KB 20480 bytes
0xF000 - 0xFFFF I/O 4 KB 4096 bytes

TOTAL: 64 KB = 65536 bytes

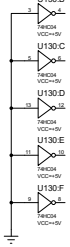
BOOT MODE:

When BOOT_ROM_EN = 0:
0x0000 - 0x7FFF BIOS mirror / boot overlay
0x8000 - 0x9FFF BIOS fixed

When BOOT_ROM_EN = 1:
0x0000 - 0x7FFF RAM
0x8000 - 0x9FFF BIOS fixed

DECODE:

RAM_SEL = 0x0000 - 0x7FFF
BIOS_SEL = 0x8000 - 0x9FFF
CART_SEL = 0xA000 - 0xEFFF
IO_SEL = 0xF000 - 0xFFFF

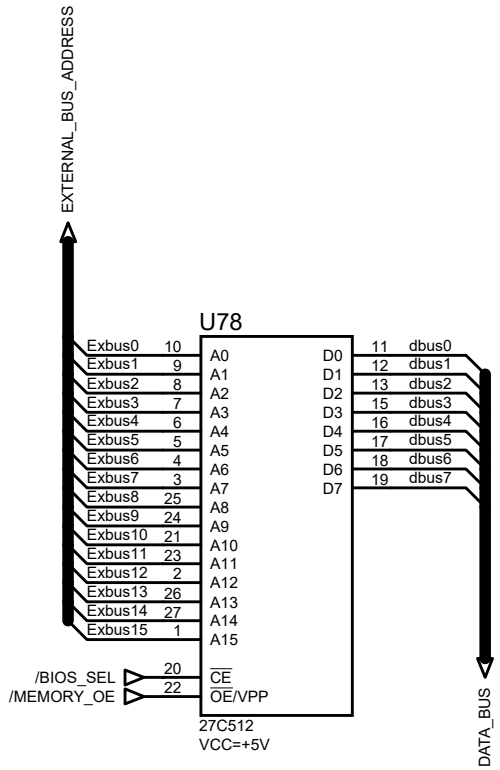


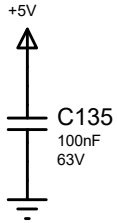
PROJECT: Wall - 8bits Computer	DATE: 13/08/2026
PAGE: Decode Memory	
FILE NAME: Hardware.pdsprj	PAGE: 5 of 17
DRAWN BY: Vinicius Jesus	REV: 1.0

WALL-8 ROM MEMORY

ADDRESS BUS: 16 bits
DATA BUS: 8 bits

0x8000 - 0x9FFF BIOS/ROM 8 KB 8192 bytes



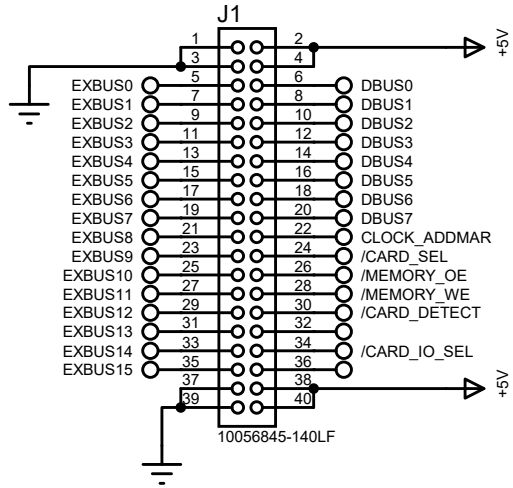


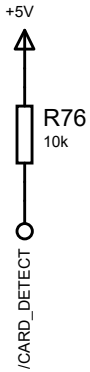
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **ROM/BIOS Memory**	
FILE NAME: **Hardware.pdsprj**	PAGE: **10 of 17**
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

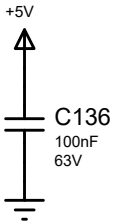
WALL-8 CARD MEMORY

ADDRESS BUS: 16 bits
DATA BUS: 8 bits

0xA000 - 0xEFFF CARD 20 KB 20.480 bytes

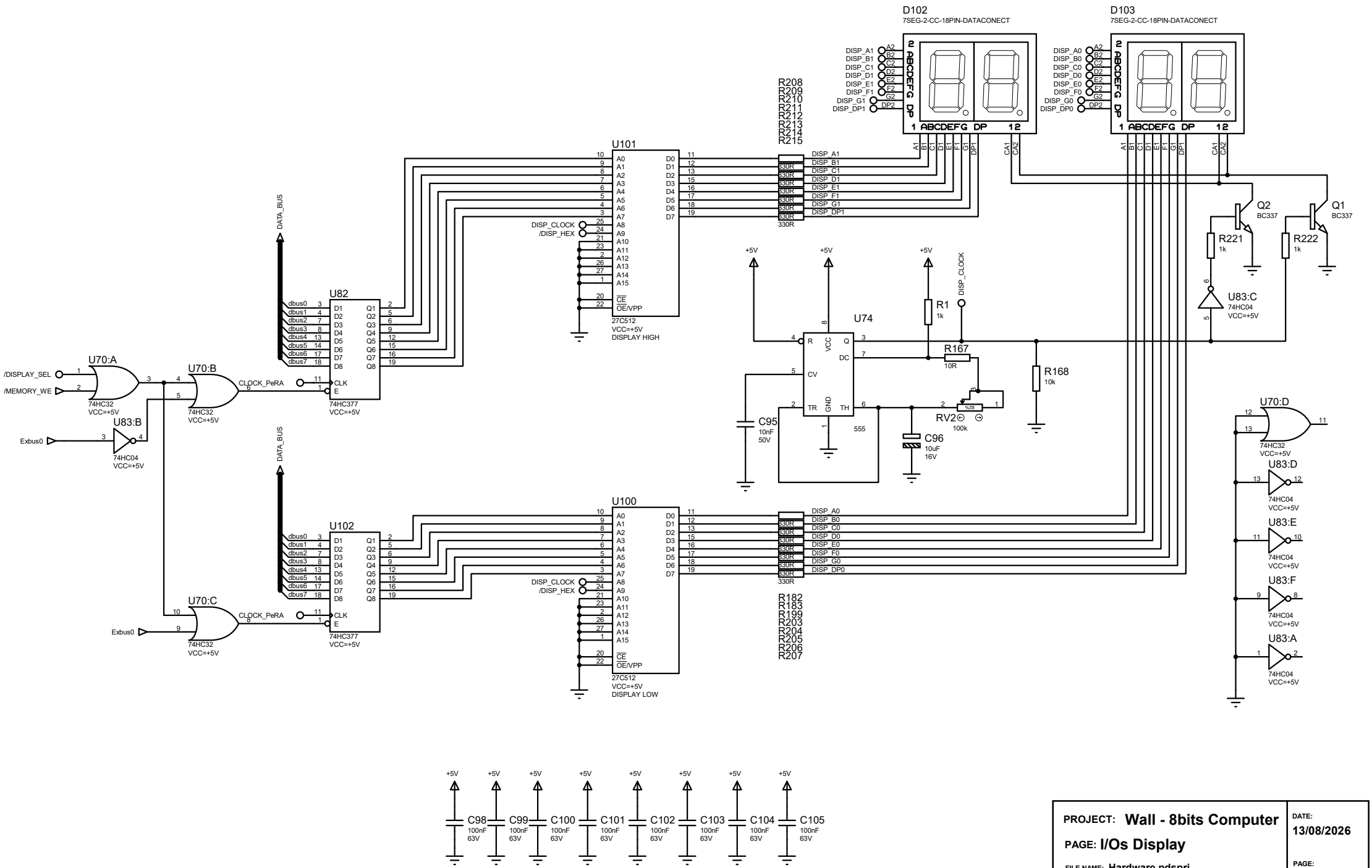






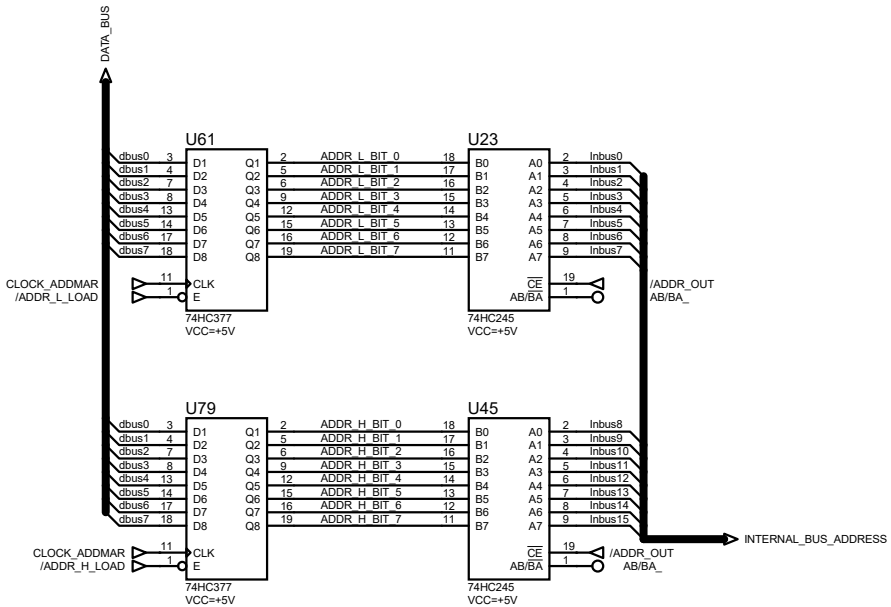
PROJECT: Wall - 8bits Computer	**DATE:** 13/08/2026
PAGE: CARD Memory	
FILE NAME: Hardware.pdsprj	**PAGE:** 11 of 17
DRAWN BY: Vinicius Jesus **REV:** 1.0	

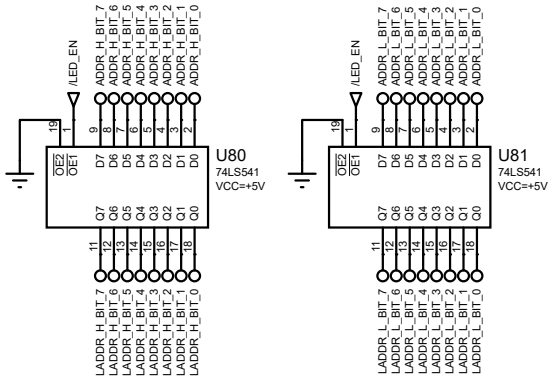
DISPLAY HEX

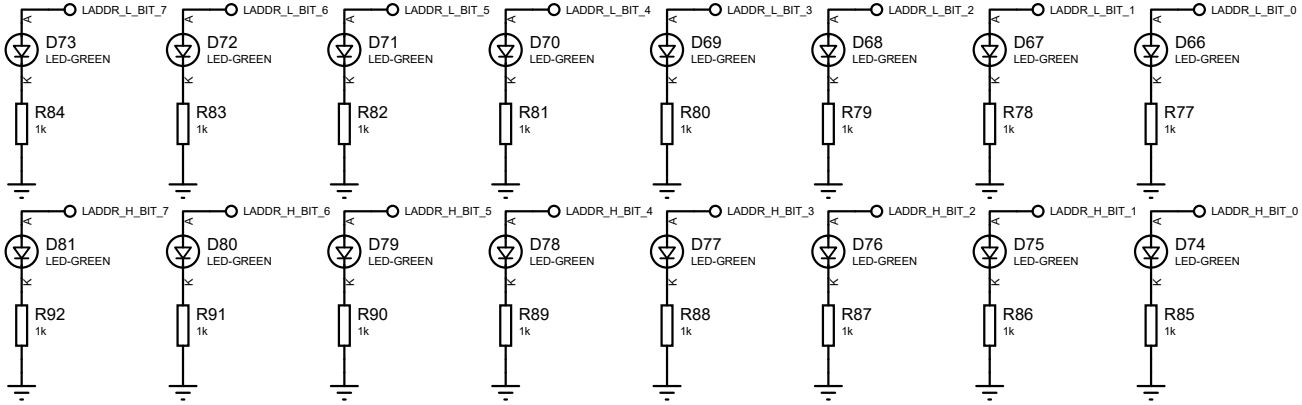


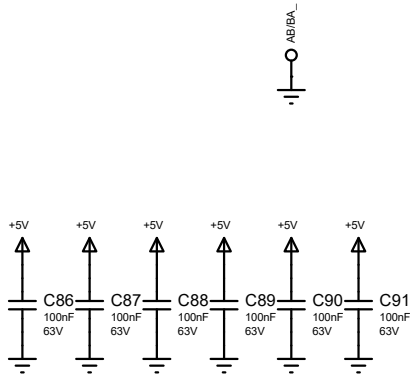
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **I/Os Display**	PAGE: **12 of 17**
FILE NAME: **Hardware.pdsprj**	
DRAWN BY: **Vinicius Jesus**	REV: **1.0**

ADDR - Address Register







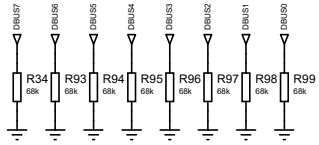


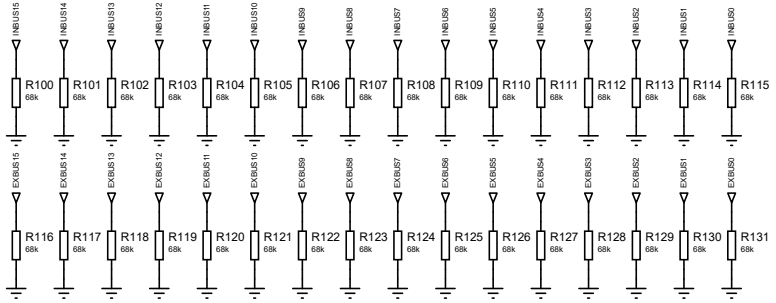
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **ADDR**	
FILE NAME: **Hardware.pdsprj**	PAGE: **13 of 17**
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

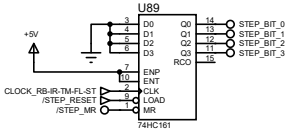
WALL-8 UNIT CONTROL

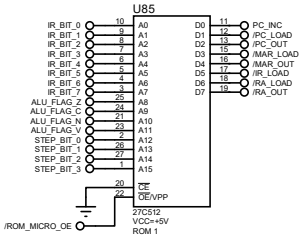
Valores de NEUTRO , (TUDO DESABILITADO)

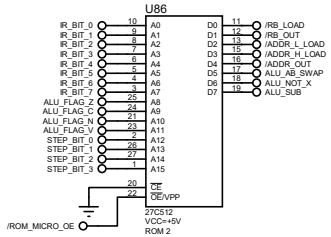
ROM 1 0xFE
ROM 2 0X1F
ROM 3 0xF8
ROM 4 0xDF
ROM 5 0x6F

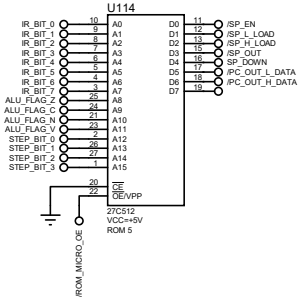


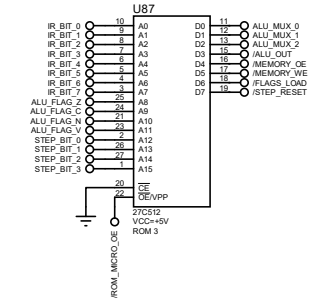


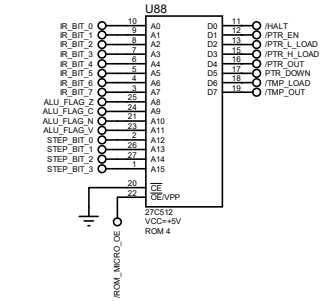


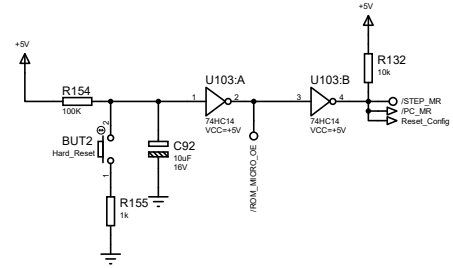


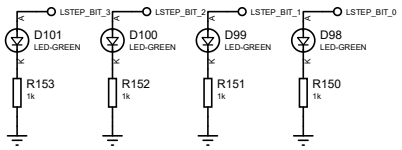


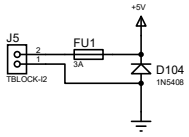


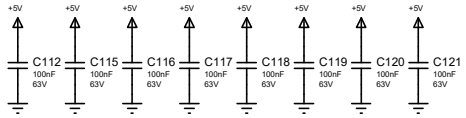


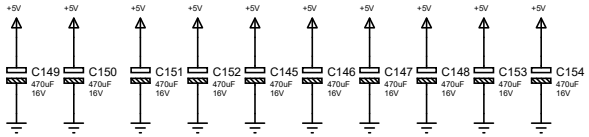


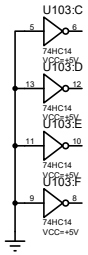


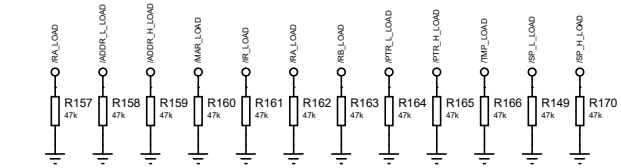








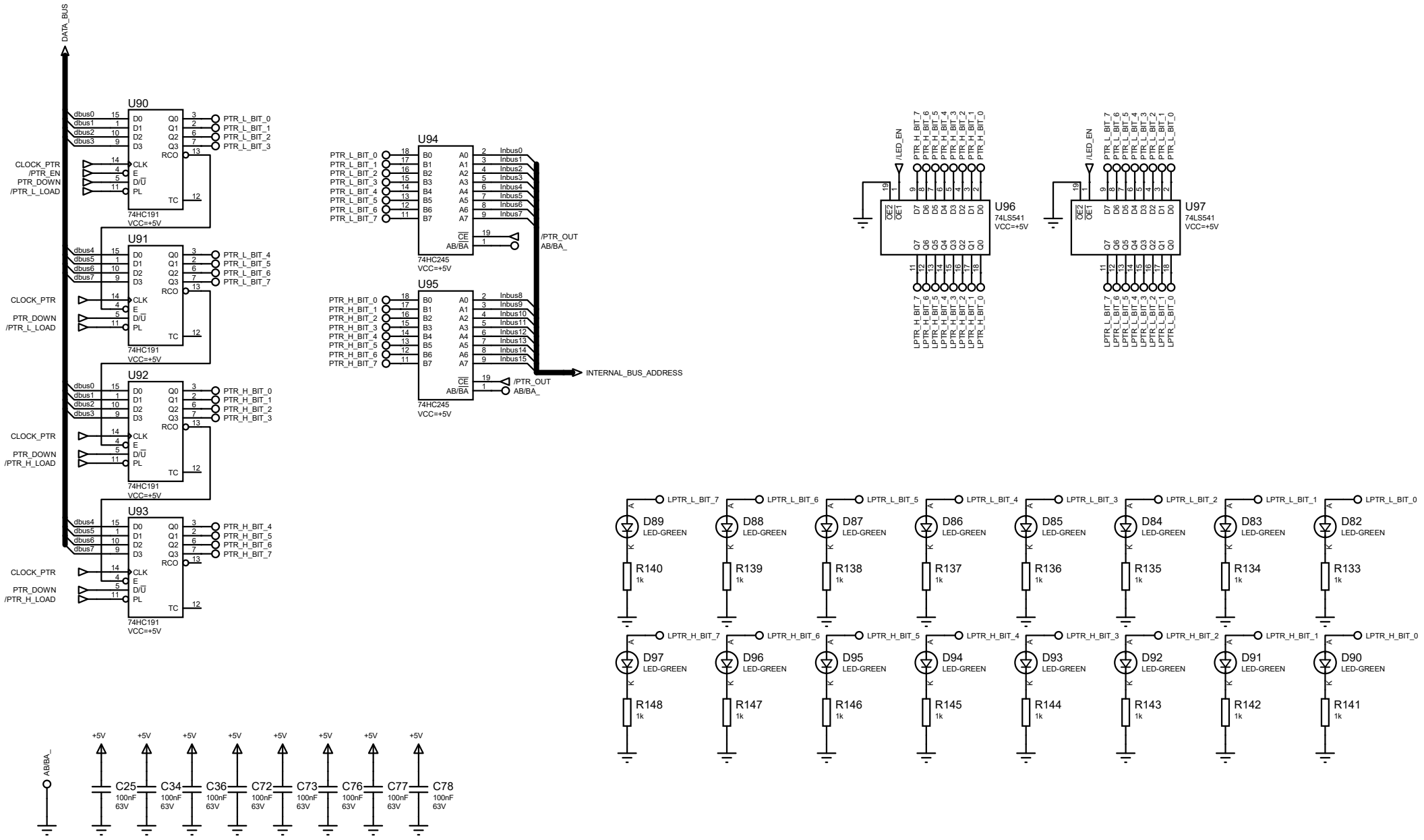






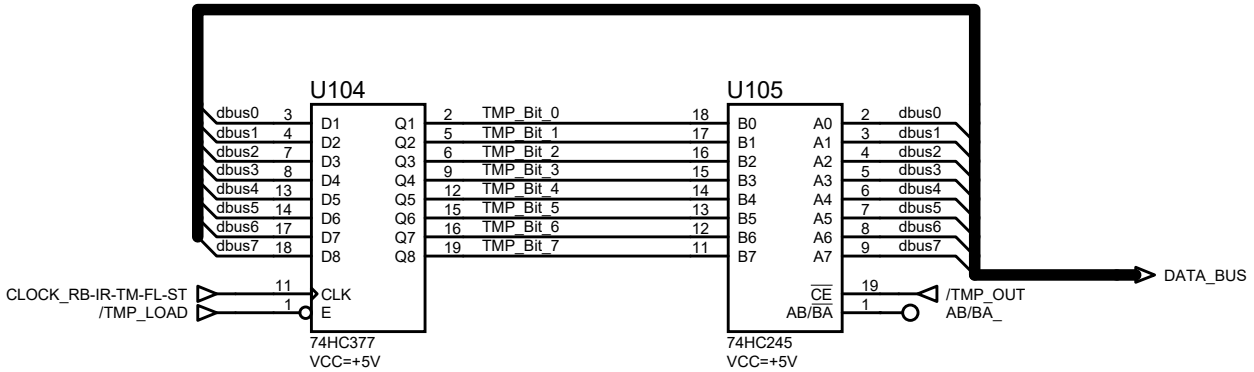
PROJECT: **Wall - 8bits Computer**	DATE: **13/08/2026**
PAGE: **Unit Control**	PAGE: **14 of 17**
FILE NAME: **Hardware.pdsprj**	
DRAWN BY: **Vinicius Jesus** REV: **1.0**	

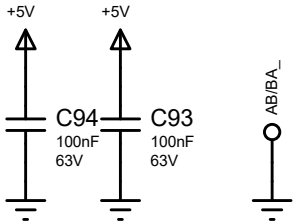
PTR - Pointer Register



PROJECT: **Wall - 8bits Computer**		DATE: **13/08/2026**
PAGE: **PTR**		
FILE NAME: **Hardware.pdsprj**		PAGE: **15 of 17**
DRAWN BY: **Vinicius Jesus**	REV: **1.0**	

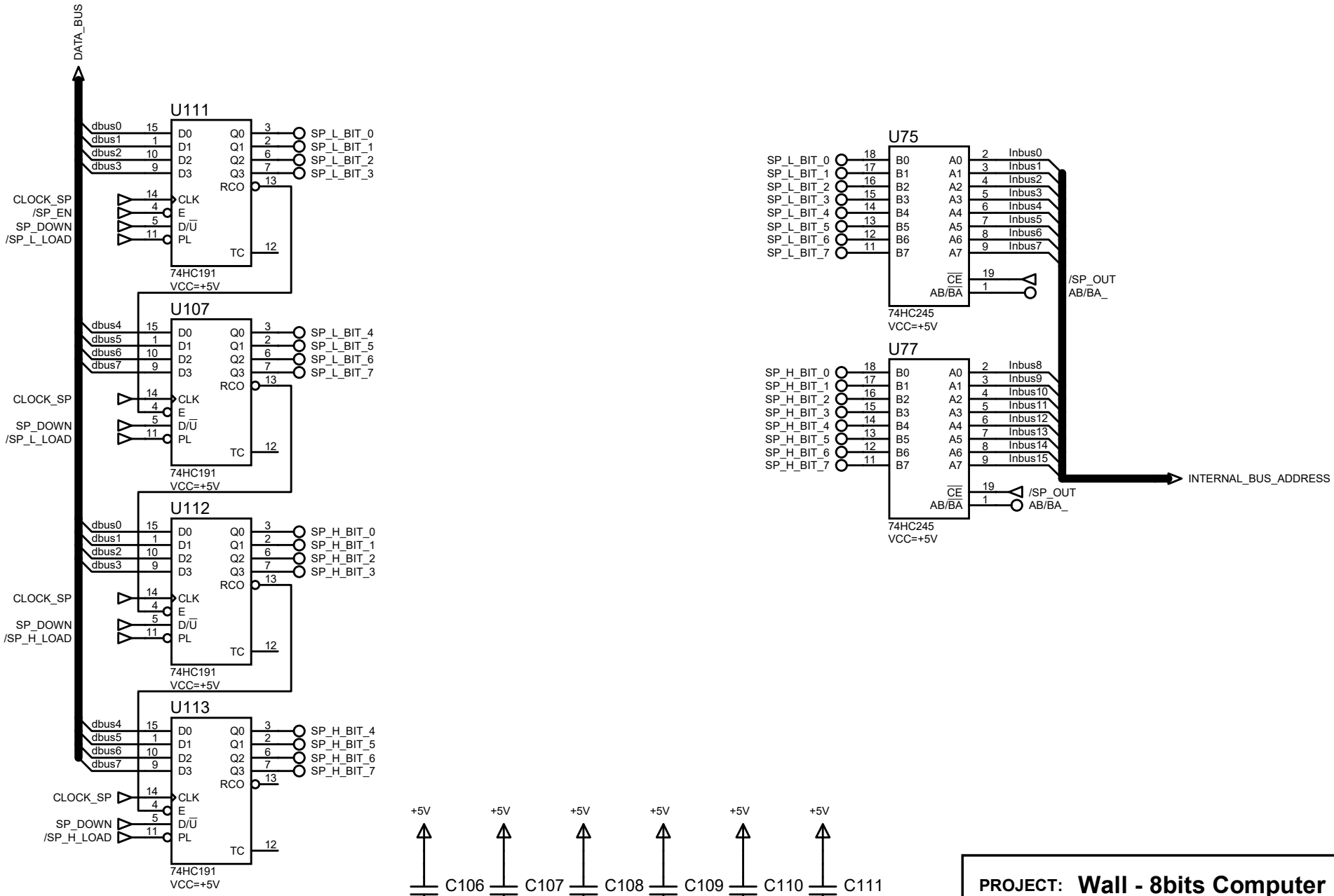
TMP - Temporary Register





PROJECT: **Wall - 8bits Computer**		DATE: **13/08/2026**
PAGE: **Registrador TMP**		
FILE NAME: **Hardware.pdsprj**		PAGE: **16 of 17**
DRAWN BY: **Vinicius Jesus**	REV: **1.0**	

SP - Stack Point



PROJECT: Wall - 8bits Computer	**DATE:** 13/08/2026
PAGE: SP - Stack Point	
FILE NAME: Hardware.pdsprj	**PAGE:** 17 of 17
DRAWN BY: Vinicius Jesus **REV:** 1.0	